UNIVERSITY OF SCIENCE AND TECHNOLOGY

Wydział Informatyki

Projekt Bazy Danych - Firma Produkcyjno Usługowa

Autorzy:
Dagmara Pasik, Mikołaj Wałek, Kacper Kulka

30 stycznia 2026

Spis treści

1. Opis projektu ... 6
2. Schemat ... 7
3. Tabele ... 8
   3.1. Tabela InventoryParts ... 8
      3.1.1. Pola tabeli ... 8
      3.1.2. Klucze ... 8
      3.1.3. Warunki integralności ... 9
   3.2. Tabela Customers ... 9
      3.2.1. Pola tabeli ... 9
      3.2.2. Klucze ... 10
      3.2.3. Warunki integralności ... 10
   3.3. Tabela Payments ... 11
      3.3.1. Pola tabeli ... 11
      3.3.2. Klucze ... 12
      3.3.3. Warunki integralności ... 12
   3.4. Tabela Products ... 13
      3.4.1. Pola tabeli ... 13
      3.4.2. Klucze ... 14
      3.4.3. Warunki integralności ... 14
   3.5. Tabela Parts ... 14
      3.5.1. Pola Tabeli ... 15
      3.5.2. Klucze ... 15
      3.5.3. Warunki integralności ... 15
   3.6. Tabela Order Execution ... 16
      3.6.1. Pola Tabeli ... 16
      3.6.2. Klucze ... 16
      3.6.3. Warunki integralności ... 16
   3.7. Tabela Shippers ... 17
      3.7.1. Pola tabeli ... 17
      3.7.2. Klucze ... 18
      3.7.3. Warunki integralności ... 18
   3.8. Tabela Suppliers ... 19
      3.8.1. Pola Tabeli ... 19
      3.8.2. Klucze ... 20
      3.8.3. Warunki integralności ... 20
   3.9. Tabela Employees ... 21
      3.9.1. Pola Tabeli ... 21
      3.9.2. Klucze ... 22
      3.9.3. Warunki integralności ... 22
   3.10. Tabela Orders ... 23
      3.10.1. Pola Tabeli ... 23
      3.10.2. Klucze ... 24
      3.10.3. Warunki integralności ... 24

3.10.4. Klucze ... 25
3.10.5. Warunki integralności ... 25
3.11. Tabela Order Details ... 25
3.11.1. Pola Tabeli ... 26
3.11.2. Klucze ... 26
3.11.3. Warunki integralności ... 26
3.12. Tabela Inventory Products ... 26
3.12.1. Pola Tabeli ... 27
3.12.2. Klucze ... 27
3.12.3. Warunki integralności ... 27
4. Funkcje ... 28
4.1. Funkcja fn_AvgOrderCompletionTime ... 28
4.1.1. Parametry ... 28
4.1.2. Zwracana wartość ... 28
4.2. Funkcja fn_EmployeeAccess ... 28
4.2.1. Paramtery ... 29
4.2.2. Zwracana wartość ... 29
4.3. Funkcja fn_LastOrderOfCustomer ... 29
4.3.1. Parametry ... 29
4.3.2. Zwracana wartość ... 29
4.4. Funkcja fn_MostFreqOrderedProduct ... 30
4.4.1. Parametry ... 30
4.4.2. Zwracana wartość ... 30
4.5. Funkcja fn_OrderTotal ... 30
4.5.1. Parametry ... 31
4.5.2. Zwracana wartość ... 31
4.6. Funkcja fn_OrderProductionTimeScalar ... 31
4.6.0.1. Parametry ... 32
4.6.0.2. Zwracana wartość ... 32
4.7. Funkcja fn_RemainingOrderAmount ... 32
4.7.1. Parametry ... 32
4.7.2. Zwracana wartość ... 32
4.8. Funkcja fn_RevenueInRange ... 33
4.8.0.1. Parametry ... 33
4.8.0.2. Zwracana wartość ... 33
4.9. Funkcja fn_OrderCount_ByCustomerID ... 34
4.9.0.1. Parametry ... 34
4.9.0.2. Zwracana wartość ... 34
4.10. Funkcja fn_EmployeesWithMostOrders ... 35
4.10.1. Parametry ... 35
4.10.2. Zwracana wartość ... 36
4.11. Funkcja fn_GetAverageOrderValue ... 36
4.11.1. Parametry ... 37
4.11.2. Zwracana wartość ... 37
4.12. Funkcja fn_GetProductRevenue ... 37

4.12.1. Parametry ... 38
4.12.2. Zwracana wartość ... 38
4.13. Funkcja fn_LeastBoughtProducts ... 38
4.13.1. Parametry ... 39
4.13.2. Zwracana wartość ... 39
5. Procedury ... 40
5.1. Procedura AddEmployee ... 40
5.2. Procedura AddSupplier ... 40
5.3. Procedura AddOrderDetails ... 41
5.4. Procedura AddInventoryProduct ... 41
5.5. Procedura AddOrder ... 42
5.6. Procedura ChangeOrderData ... 43
5.7. Procedura AssignShippers ... 44
5.8. Procedura AddCustomer ... 45
5.9. Procedura ChangeCustomerData ... 46
5.10. Procedura ChangeCustomerContact ... 47
5.11. Procedura ChangeEmployeeData ... 48
5.12. Procedura ChangeEmployeeContact ... 49
5.13. Procedura ChangeIsActiveEmployee ... 50
5.14. Procedura ChangeSupplierData ... 51
5.15. Procedura ChangeSupplierContact ... 52
5.16. Procedura AddProduct ... 52
5.17. Procedura ChangeProductData ... 53
5.18. Procedura AddInventoryPart ... 54
5.19. Procedura AddPart ... 54
5.20. Procedura AddPayment ... 55
5.21. Procedura AddShipper ... 56
5.22. Prodcedura ChangeIsActiveShipper ... 57
5.23. Procedura ChangePartData ... 58
5.24. Procedura ChangeShipperContact ... 59
5.25. Procedura ChangeShipperData ... 60
6. Triggery ... 61
6.1. Trigger trg_BlockInactiveEmployees ... 61
6.2. Trigger trg_InventoryParts_AutoAddParts ... 62
6.3. Trigger trg_Orders_PredictOrderExecution ... 63
6.4. Trigger trg_Payments_UpdatePaymentStatus ... 65
6.5. Trigger trg_Orders_AddOrderDate ... 66
6.6. Trigger trg_Orders_InventoryDecrement ... 67
6.7. Trigger trg_InventoryProducts_AutoAddStock ... 68
6.8. Trigger trg_OrdersBlockInactiveShippers ... 69
6.9. Trigger trg_Payments_OrderStatus2_To3_OnFullyPaid ... 70
7. Widoki ... 71
7.1. Widok v_ActiveEmployeesOrders ... 71
7.2. Widok v_CustomerRemainingAmounts ... 71
7.3. Widok v_InventoryProductsStockInfo ... 72

7.4. Widok v_OrderBasic ... 72
7.5. Widok v_OrderValue ... 73
7.6. Widok v_CustomerOrderHistory ... 74
7.7. Widok v_OrderSchedule ... 75
7.8. Widok v_ProductSalesStatistic ... 76
8. Uprawnienia, użytkownicy systemu i funkcje użytkowników ... 76
8.1. Rola admin (administrator) ... 77
8.1.1. Użytkownik systemu ... 77
8.2. Rola employee (pracownik) ... 77
8.2.1. Użytkownik systemu ... 77
8.2.2. Funkcje użytkownika ... 78
8.3. Zakres uprawnień ... 78
8.4. Rola warehouse (magazyn) ... 78
8.4.1. Użytkownik systemu ... 78
8.4.2. Funkcje użytkownika ... 79
8.4.3. Zakres uprawnień ... 79
8.5. Rola finance (finanse) ... 79
8.5.1. Użytkownik systemu ... 79
8.5.2. Funkcje użytkownika ... 80
8.5.3. Zakres uprawnień ... 80
8.6. Rola reporting (raportowanie / podgląd) ... 80
8.6.1. Użytkownik systemu ... 80
8.6.2. Funkcje użytkownika ... 81
8.6.3. Zakres uprawnień ... 81
8.7. Rola endUser (użytkownik) ... 81
8.8. Funkcje użytkownika ... 82
8.8.1. Zakres uprawnień ... 82
8.9. Rola logistics (logistyka) ... 82
8.9.1. Funkcje użytkownika ... 83
8.9.2. Zakres uprawnień ... 83

1. Opis projektu

Projekt dotyczy stworzenia bazy danych dla firmy produkcyjno-usługowej zajmującej się wytwarzaniem oraz sprzedażą mebli przeznaczonych do wyposażenia pomieszczeń z infrastrukturą komputerową, takich jak sale dydaktyczne oraz biura. Oferta firmy obejmuje między innymi krzesła, biurka, stoły, fotele, stojaki na projektory oraz tablice interaktywne.

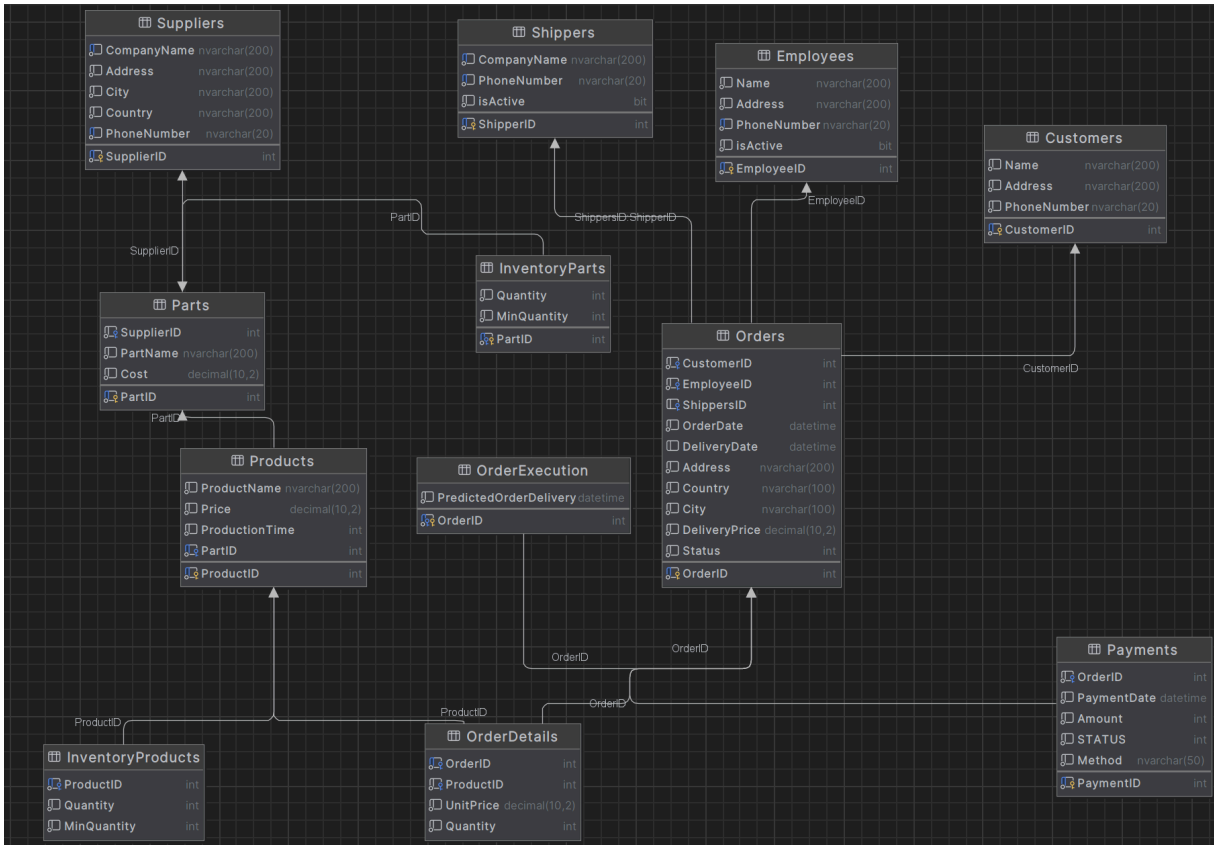
System wspiera proces produkcji poprzez definiowanie struktury produktów, określającej z jakich części składa się dany wyrób oraz jaki czas jest potrzebny na jego wykonanie. Na tej podstawie możliwe jest planowanie produkcji oraz realizacja zamówień w sytuacji, gdy produktu nie ma aktualnie w magazynie.

Część sprzedażowa umożliwia obsługę klientów, składanie zamówień, rejestrowanie płatności oraz przyznawanie rabatów. System pozwala także na monitorowanie stanów magazynowych produktów gotowych i części.

Projekt obejmuje również przygotowanie zestawu raportów analitycznych dotyczących kosztów produkcji, sprzedaży produktów, stanów magazynowych oraz historii zamówień klientów. Dane te wspierają podejmowanie decyzji zarządczych oraz planowanie dalszej działalności firmy.

Celem projektu jest stworzenie spójnego systemu wspomagającego procesy produkcyjne i sprzedażowe oraz zapewniającego przejrzysty dostęp do informacji niezbędnych w codziennym funkcjonowaniu przedsiębiorstwa.

2. Schemat



Rysunek 1: Rys. 1. Diagram bazy danych

3. Tabele

3.1. Tabela InventoryParts

```sql
CREATE TABLE dbo.InventoryParts
(
    PartID INT NOT NULL,
    Quantity INT NOT NULL
        CONSTRAINT CK_InventoryParts_Quantity_NonNegative CHECK (Quantity >= 0),
    MinQuantity INT NOT NULL
        CONSTRAINT CK_InventoryParts_MinQuantity_NonNegative CHECK (MinQuantity >= 0),

    CONSTRAINT PK_InventoryParts PRIMARY KEY (PartID),
    CONSTRAINT FK_InventoryParts FOREIGN KEY (PartID) REFERENCES dbo.Parts(PartID)
);
```

Rysunek 2: Struktura tabeli InventoryParts

Tabela **InventoryParts** przechowuje informacje o aktualnych stanach magazynowych części wykorzystywanych w procesie produkcji. Każdy rekord odpowiada jednej części oraz określa jej bieżącą ilość w magazynie, a także minimalny poziom zapasu.

3.1.1. Pola tabeli

- **PartID (INT)** – identyfikator części. Jest jednocześnie kluczem głównym tabeli oraz kluczem obcym wskazującym na tabelę **Parts**. Zapewnia jednoznaczną identyfikację elementu magazynowego.
- **Quantity (INT)** – aktualna liczba sztuk danej części dostępnych w magazynie.
- **MinQuantity (INT)** – minimalna wymagana liczba sztuk części. Wartość ta służy do kontroli zapasów oraz sygnalizowania potrzeby uzupełnienia magazynu.

3.1.2. Klucze

- Klucz główny: **PartID**
- Klucz obcy: **PartID → Parts(PartID)**

3.1.3. Warunki integralności

Zdefiniowane zostały następujące ograniczenia typu CHECK:

- **Quantity ≥ 0** – zabrania wprowadzania ujemnych stanów magazynowych.
- **MinQuantity ≥ 0** – wymusza nieujemny minimalny poziom zapasu.

Dodatkowo logika aplikacyjna zapewnia, że wartość **Quantity** jest zawsze większa od **MinQuantity**, co gwarantuje poprawność danych magazynowych.

3.2. Tabela Customers

```sql
CREATE TABLE dbo.Customers
(
    CustomerID  INT IDENTITY(1, 1) NOT NULL CONSTRAINT PK_Customers PRIMARY KEY,
    Name        NVARCHAR(200) NOT NULL,
    Address     NVARCHAR(200) NOT NULL,
    PhoneNumber NVARCHAR(20)  NOT NULL
        CONSTRAINT CK_Customers_Correct_Phone
        CHECK (
            PhoneNumber LIKE '+[0-9]%'
            AND PhoneNumber NOT LIKE '%[^0-9+]%'
            AND LEN(PhoneNumber) BETWEEN 10 AND 16
        )
);
```

Rysunek 3: Struktura tabeli Customers

Tabela **Customers** przechowuje informacje o klientach składających zamówienia w systemie. Każdy rekord identyfikuje jednego klienta oraz zawiera podstawowe dane kontaktowe wykorzystywane w procesie obsługi zamówień i komunikacji.

3.2.1. Pola tabeli

- **CustomerID (INT)** – unikalny identyfikator klienta. Klucz główny tabeli, generowany automatycznie (IDENTITY).
- **Name (NVARCHAR(200))** – nazwa klienta (osoba lub firma). Pole wymagane.

- **Address (NVARCHAR(200))** – adres klienta. Pole wymagane.
- **PhoneNumber (NVARCHAR(20))** – numer telefonu klienta. Pole wymagane.

3.2.2. Klucze

- Klucz główny: **CustomerID**

3.2.3. Warunki integralności

- **NOT NULL** – pola **Name**, **Address** oraz **PhoneNumber** są wymagane.
- CHECK: **CK_Customers_Correct_Phone** – wymusza poprawny format numeru telefonu:
  - numer zaczyna się od znaku + i cyfry,
  - zawiera wyłącznie znaki + oraz cyfry,
  - długość numeru mieści się w przedziale 10–16 znaków.

3.3. Tabela Payments

```sql
create table Payments
(
    PaymentID   int identity
        constraint PK_Payments
            primary key,
    OrderID     int             not null
        constraint FK_Payments_Orders
            references Orders,
    PaymentDate datetime        not null,
    Amount      decimal(10, 2) not null
        constraint CK_Payments_Amount_NonNegative
            check ([Amount] >= 0)
        constraint CK_Payments_Amount_NonNegative
            check ([Amount] >= 0),
    Status      int             not null,
    Method      nvarchar(50)    not null
)
go
```

Rysunek 4: Struktura tabeli Payments

Tabela **Payments** przechowuje informacje o płatnościach przypisanych do zamówień. Rejestruje datę płatności, kwotę, status oraz metodę płatności, co pozwala na kontrolę rozliczeń oraz analizę sprzedaży.

3.3.1. Pola tabeli

- **PaymentID (INT)** – unikalny identyfikator płatności. Klucz główny tabeli, generowany automatycznie (IDENTITY).

- **OrderID (INT)** – identyfikator zamówienia, którego dotyczy płatność. Pole wymagane.
- **PaymentDate (DATETIME)** – data i czas zarejestrowania płatności. Pole wymagane.
- **Amount (DECIMAL(10,2))** – kwota płatności. Pole wymagane.
- **Status (INT)** – status płatności: **0 – nieopłacone**, **1 – opłacone (w pełni)**. Pole wymagane.
- **Method (NVARCHAR(50))** – metoda płatności (np. przelew, karta, BLIK). Pole wymagane.

3.3.2. Klucze

- Klucz główny: **PaymentID**
- Klucz obcy: **OrderID → Orders(OrderID)**

3.3.3. Warunki integralności

- **NOT NULL** – wszystkie pola są wymagane.
- CHECK: **CK_Payments_Amount_NonNegative** – wymusza nieujemną kwotę płatności (**Amount ≥ 0**).
- Spójność referencyjna: nie można dodać płatności dla nieistniejącego zamówienia.

3.4. Tabela Products

```sql
create table Products
(
    ProductID      int identity
        primary key,
    ProductName    nvarchar(200)  not null,
    Price          decimal(10, 2) not null
        constraint CK_Products_Price_NonNegative
            check ([Price] >= 0),
    ProductionTime int            not null
        constraint CK_Products_ProductionTime_Positive
            check ([ProductionTime] > 0),
    PartID         int            not null
        constraint FK_Products_Parts
            references Parts
)
```

Rysunek 5: Struktura tabeli Products

Tabela **Products** przechowuje informacje o produktach oferowanych przez firmę. Zawiera nazwę produktu, cenę, czas produkcji oraz powiązanie z częścią, z której produkt jest wytwarzany (w aktualnym uproszczeniu: produkt składa się z jednej części).

3.4.1. Pola tabeli

- **ProductID (INT)** – unikalny identyfikator produktu. Klucz główny tabeli, generowany automatycznie (IDENTITY).
- **ProductName (NVARCHAR(200))** – nazwa produktu. Pole wymagane.
- **Price (DECIMAL(10,2))** – cena produktu. Pole wymagane.
- **ProductionTime (INT)** – czas potrzebny na wytworzenie produktu (np. w dniach). Pole wymagane.

- **PartID (INT)** – identyfikator części powiązanej z produktem. Pole wymagane.

3.4.2. Klucze

- Klucz główny: **ProductID**
- Klucz obcy: **PartID → Parts(PartID)**

3.4.3. Warunki integralności

- **NOT NULL** – pola **ProductName**, **Price**, **ProductionTime** oraz **PartID** są wymagane.
- CHECK: **CK_Products_Price_NonNegative** – wymusza nieujemną cenę (**Price ≥ 0**).
- CHECK: **CK_Products_ProductionTime_Positive** – wymusza dodatni czas produkcji (**ProductionTime > 0**).
- Spójność referencyjna: nie można przypisać produktu do nieistniejącej części.

3.5. Tabela Parts

```sql
CREATE TABLE dbo.Parts (
    PartID INT IDENTITY(1, 1) NOT NULL
        CONSTRAINT PK_Parts PRIMARY KEY,

    SupplierID INT NOT NULL
        CONSTRAINT FK_Parts_Suppliers
        FOREIGN KEY REFERENCES dbo.Suppliers(SupplierID),

    PartName NVARCHAR(200) NOT NULL,
    Cost DECIMAL(10, 2) NOT NULL
        CONSTRAINT CK_Parts_Cost_NonNegative
        CHECK (Cost >= 0)

);
```

Rysunek 6: Struktura tabeli Parts

Tabela **Parts** zawiera wykaz wszystkich komponentów, które firma pozyskuje od dostawców w celu dalszego montażu własnych produktów. Przechowuje ona dane o półproduktach i elementach montażowych, co pozwala na precyzyjnie zarządzanie zasobami niezbędnymi do realizacji procesów produkcyjnych.

3.5.1. Pola Tabeli

- **PartID (INT)** - unikalny identyfikator części. Klucz główny tabeli, generowany automatycznie (IDENTITY).
- **SupplierID (INT)** - identyfikator dostawcy odpowiedzialnego za dostarczenie części. Klucz obcy, pole wymagane.
- **PartName (NVARCHAR(200))** - nazwa techniczna części. Pole wymagane.
- **Cost (DECIMAL(10, 2))** - koszt zakupu komponentu. Pole wymagane.

3.5.2. Klucze

- Klucz główny: **PartID**
- Klucz obcy: **SupplierID** -> **Suppliers(SupplierID)**

3.5.3. Warunki integralności

- **NOT NULL** - pola **PartID**, **SupplierID**, **PartName** oraz **Cost** są wymagane.
- CHECK: **CK_Parts_Cost_NonNegative** - wymusza, aby koszt zakupu był wartośćią nieujemną (**Cost ≥ 0**)
- Spójność referencyjna: każda część musi być przypisana do istniejącego dostawcy w systemie.

3.6. Tabela Order Execution

```sql
CREATE TABLE dbo.OrderExecution (
    OrderID INT NOT NULL
        CONSTRAINT PK_OrderExecution PRIMARY KEY (OrderID),
        CONSTRAINT FK_OrderExecution_Orders FOREIGN KEY (OrderID) REFERENCES dbo.Orders(OrderID),
    PredictedOrderDelivery DATETIME NOT NULL
);
```

Rysunek 7: Struktura tabeli Order Execution

Tabela **Order Execution** służy do monitorowania procesu realizacji zamówień poprzez przechowywanie terminów dostawy. Pozwala ona na synchronizację działu logistyki oraz informowanie klientów o planowanym czasie otrzymania gotowych produktów.

3.6.1. Pola Tabeli

- **OrderID (INT)** - identyfikator zamówienia, którego dotyczy estymacja dostawy. Pełni funkcję klucza głównego.
- **PredictedOrderDelivery (DATETIME)** - przewidywana data i czas dostarczenia zamówienia do klienta. Pole wymagane.

3.6.2. Klucze

- Klucz główny: **OrderID**
- Klucz obcy: **OrderID** -> **Orders(OrderID)**

3.6.3. Warunki integralności

- **NOT NULL** - wszystkie pola w tabeli są wymagane.
- Spójność referencyjna: każdy rekord w tej tabeli musi odpowiadać istniejącemu zamówieniu w tabeli **Orders**.
- Unikalność: ze względu na klucz główny na polu OrderID, dla każdego zamówienia może istnieć tylko jedna przewidywana data dostawy.

3.7. Tabela Shippers

```sql
CREATE TABLE dbo.Shippers
(
    ShipperID INT IDENTITY(1, 1) NOT NULL
        CONSTRAINT PK_Shippers PRIMARY KEY,
    CompanyName NVARCHAR(200) NOT NULL
        CONSTRAINT UQ_Shippers_CompanyName UNIQUE,
    PhoneNumber NVARCHAR(20) NOT NULL
        CONSTRAINT UQ_Shippers_PhoneNumber UNIQUE,
        CONSTRAINT CK_Shippers_Correct_Phone
        CHECK (
            PhoneNumber LIKE '+[0-9]%'
            AND PhoneNumber NOT LIKE '%[^0-9+]%'
            AND len(PhoneNumber) BETWEEN 10 and 16
            ),
    isActive BIT NOT NULL
);
```

Rysunek 8: Struktura tabeli Shippers

Tabela **Shippers** zawiera wykaz firm kurierskich współpracujących z firmą w celu dostarczenia gotowych produktów do klientów. Pozwala ona na zarządzanie bazą kontaktową przewoźników oraz kontrolowanie ich dostępności w systemie.

3.7.1. Pola tabeli

- **ShipperID (INT)** - unikalny identyfikator firmy kurierskiej. Klucz główny tabeli, generowany automatycznie (IDENTITY).
- **CompanyName (NVARCHAR(200))** - pełna nazwa firmy przewoźnika. Pole wymagane i unikalne.

- **PhoneNumber (NVARCHAR(20))** - numer telefonu kontaktowego do przewoźnika. Pole wymagane i unikalne.
- **isActive (BIT)** - flaga określająca, czy dany przewoźnik jest aktualnie aktywny i może realizować dostawy. Pole wymagane.

3.7.2. Klucze

- Klucz główny: **ShipperID**

3.7.3. Warunki integralności

- **NOT NULL** - wszystkie pola w tabeli są wymagane.
- UNIQUE: **UQ_Shippers_CompanyName** - nazwa firmy się nie może powtarzać.
- UNIQUE: **UQ_Shippers_PhoneNumber** - dany numer telefonu może być przypisany tylko do jednego spedytora.
- CHECK: **CK_Shippers_Correct_Phone** – wymusza poprawny format numeru telefonu: musi zaczynać się od znaku „+”, zawierać wyłącznie cyfry (poza znakiem plusa) oraz mieć długość od 10 do 16 znaków.

3.8. Tabela Suppliers

```sql
CREATE TABLE dbo.Suppliers (
    SupplierID INT IDENTITY(1, 1) NOT NULL
        CONSTRAINT PK_Suppliers PRIMARY KEY,
    CompanyName NVARCHAR(200) NOT NULL
        CONSTRAINT UQ_Suppliers_CompanyName UNIQUE,
    Address NVARCHAR(200) NOT NULL,
    City NVARCHAR(200) NOT NULL,
    Country NVARCHAR(200) NOT NULL,
    PhoneNumber NVARCHAR(20)  NOT NULL
        CONSTRAINT UQ_Suppliers_PhoneNumber UNIQUE,
        CONSTRAINT CK_Suppliers_Correct_Phone
        CHECK (
            PhoneNumber LIKE '+[0-9]%'
            AND PhoneNumber NOT LIKE '%[^0-9+]%'
            AND LEN(PhoneNumber) BETWEEN 10 AND 16
            )

);
```

Rysunek 9: Struktura tabeli Suppliers

Tabela **Suppliers** przechowuje szczegółowe informacje o podmiotach zewnętrznych dostarczających komponenty niezbędne do montażu produktów. Rejestruje dane teleadresowe dostawców, co umożliwia sprawne zarządzanie zamówieniami surowców oraz utrzymywanie relacji biznesowych.

3.8.1. Pola Tabeli

- **SupplierID (INT)** – unikalny identyfikator dostawcy. Klucz główny tabeli, generowany automatycznie (IDENTITY).

- **CompanyName (NVARCHAR(200))** – pełna nazwa firmy dostarczającej części. Pole wymagane i unikalne.
- **Address (NVARCHAR(200))** – adres siedziby dostawcy (ulica i numer). Pole wymagane.
- **City (NVARCHAR(200))** – miejscowość, w której znajduje się firma dostawcy. Pole wymagane.
- **Country (NVARCHAR(200))** – kraj pochodzenia dostawcy. Pole wymagane.
- **PhoneNumber (NVARCHAR(20))** – numer telefonu kontaktowego do dostawcy. Pole wymagane i unikalne.

3.8.2. Klucze

- Klucz główny: SupplierID

3.8.3. Warunki integralności

- **NOT NULL** – wszystkie pola w tabeli są wymagane.
- UNIQUE: **UQ_Suppliers_CompanyName** – nazwa dostawcy musi być unikalna w skali całego systemu.
- UNIQUE: **UQ_Suppliers_PhoneNumber** – dany numer telefonu nie może być przypisany do więcej niż jednego dostawcy.
- CHECK: **CK_Suppliers_Correct_Phone** – wymusza poprawny format numeru telefonu: musi zaczynać się od znaku „+”, składać się wyłącznie z cyfr (poza znakiem plusa) oraz mieć długość od 10 do 16 znaków.

3.9. Tabela Employees

```sql
CREATE TABLE dbo.Employees
(
    EmployeeID INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_Employees PRIMARY KEY,
    Name NVARCHAR(200) NOT NULL,
    Address NVARCHAR(200) NOT NULL,
    PhoneNumber NVARCHAR(20) NOT NULL
        CONSTRAINT UQ_Employees_PhoneNumber UNIQUE
        CHECK (
            PhoneNumber LIKE '+[0-9]%'
            AND PhoneNumber NOT LIKE '%[^0-9+]%'
            AND LEN(PhoneNumber) BETWEEN 10 AND 16
            ),
    isActive BIT NOT NULL
);
```

Rysunek 10: Struktura tabeli Employees

Tabela **Employees** przechowuje wszystkie informacje o pracownikach firmy.

3.9.1. Pola Tabeli

- **EmployeeID (INT)** - unikalny identyfikator pracownika. Klucz główny tabeli, generowany automatycznie (IDENTITY).
- **Name (NVARCHAR(200))** - imię i nazwisko pracownika. Pole wymagane.
- **Address (NVARCHAR(200))** - adres pracownika (ulica i numer). Pole wymagane.
- **PhoneNumber (NVARCHAR(20))** - numer telefonu kontaktowego do pracownika. Pole wymagane i unikalne.
- **isActive (BIT)** - status pracownika. Pole wymagane.

3.9.2. Klucze

- Klucz główny: EmployeeID

3.9.3. Warunki integralności

- **NOT NULL** – wszystkie pola w tabeli są wymagane.
- UNIQUE: **UQ_Employees_PhoneNumber** – numer telefonu musi być unikalny w skali całego systemu.
- CHECK: **CK_Employees_Correct_Phone** – wymusza poprawny format numeru telefonu: musi zaczynać się od znaku „+”, składać się wyłącznie z cyfr (poza znakiem plusa) oraz mieć długość od 10 do 16 znaków.

3.10. Tabela Orders

```sql
create table Orders
(
    OrderID       int identity
        constraint PK_Orders
            primary key,
    CustomerID    int                                   not null
        constraint FK_Orders_Customers
            references Customers,
    EmployeeID    int                                   not null
        constraint FK_Orders_Employees
            references Employees,
    ShippersID    int
        constraint FK_Orders_Shippers
            references Shippers,
    OrderDate     datetime                              not null,
    DeliveryDate  datetime,
    Address       nvarchar(200)                         not null,
    Country       nvarchar(100)                         not null,
    City          nvarchar(100)                         not null,
    DeliveryPrice decimal(10, 2)
        constraint DF_Orders_DeliveryPrice default 14.99 not null
        constraint CK_Orders_DeliveryPrice
            check ([DeliveryPrice] >= 0),
    Status        int
        constraint DF_Orders_Status default 2           not null
)
go
```

Rysunek 11: Struktura tabeli Orders

Tabela **Orders** przechowuje kluczowe informacje o zamówieniach w systemie. Jej uzupełnieniem jest tabela **Order Details**.

3.10.1. Pola Tabeli

- **OrderID (INT)** - unikalny identyfikator zamówienia. Klucz główny tabeli, generowany automatycznie (IDENTITY).

- **CustomerID (INT)** - klucz obcy, identyfikator klienta. Pole wymagane.
- **EmployeeID (INT)** - klucz obcy, identyfikator pracownika. Pole wymagane.
- **ShippersID (INT)** - klucz obcy, identyfikator przewoźnika. Pole opcjonalne (może być NULL).
- **OrderDate (DATETIME)** - data złożenia/zarejestrowania zamówienia. Pole wymagane.
- **DeliveryDate (DATETIME)** - data dostawy/realizacji zamówienia. Pole opcjonalne.
- **Address (NVARCHAR(200))** - adres dostawy. Pole wymagane.
- **Country (NVARCHAR(100))** - kraj dostawy. Pole wymagane.
- **City (NVARCHAR(100))** - miasto dostawy. Pole wymagane.
- **DeliveryPrice (DECIMAL(10,2))** - koszt dostawy. Pole wymagane (domyślnie 14.99).
- **Status (INT)** - status zamówienia (domyślnie 2). Pole wymagane.

3.10.2. Klucze

- Klucz główny: **OrderID**
- Klucz obcy: **CustomerID → Customers(CustomerID)**
- Klucz obcy: **EmployeeID → Employees(EmployeeID)**
- Klucz obcy: **ShippersID → Shippers(ShipperID)**

3.10.3. Warunki integralności

- **NOT NULL** – wszystkie pola w tabeli oprócz **ShippersID** oraz **DeliveryDate** są wymagane.
- DEFAULT: **DF_Orders_DeliveryPrice** – domyślna wartość kosztu dostawy wynosi 14.99.

- CHECK: **CK_Orders_DeliveryPrice** – koszt dostawy musi być nieujemny (**DeliveryPrice ≥ 0**).
- DEFAULT: **DF_Orders_Status** – domyślny status zamówienia wynosi 2.

3.10.4. Klucze

- Klucz główny: OrderID
- Klucz obcy: CustomerID -> Customers(CustomerID)
- Klucz obcy: EmployeeID -> Employees(EmployeeID)
- Klucz obcy: ShippersID -> Shippers(ShipperID)

3.10.5. Warunki integralności

- **NOT NULL** – wszystkie pola w tabeli oprócz DeliveryDate są wymagane.
- CHECK: **CK_Employees_Correct_Phone** – cena wysyłki musi być nieujemna.

3.11. Tabela Order Details

```sql
CREATE TABLE dbo.OrderDetails(
    OrderID   INT NOT NULL
        CONSTRAINT FK_OrderDetails_Orders
        FOREIGN KEY REFERENCES dbo.Orders(OrderID),
    ProductID INT NOT NULL
        CONSTRAINT FK_OrderDetails_Products
        FOREIGN KEY REFERENCES dbo.Products(ProductID),
    UnitPrice DECIMAL(10,2) NOT NULL
        CONSTRAINT CK_OrderDetails_UnitPrice CHECK (UnitPrice >= 0),
    Quantity  INT NOT NULL
        CONSTRAINT CK_OrderDetails_Quantity CHECK (Quantity > 0),
    CONSTRAINT PK_OrderDetails PRIMARY KEY (OrderID, ProductID)
);
```

Rysunek 12: Struktura tabeli Order Details

Tabela **Order Details** przechowuje szczegóły dotyczące zamówienia. Jest uzupełnieniem tabeli **Orders** i zawiera pozycje zamówienia wraz z ceną jednostkową oraz ilością.

3.11.1. Pola Tabeli

- **OrderID (INT)** - klucz obcy, identyfikator zamówienia. Pole wymagane.
- **ProductID (INT)** - klucz obcy, identyfikator produktu. Pole wymagane.
- **UnitPrice (DECIMAL(10,2))** - cena jednostkowa produktu. Pole wymagane.
- **Quantity (INT)** - liczba sztuk zamówionego produktu. Pole wymagane.

3.11.2. Klucze

- Klucz główny: **(OrderID, ProductID)**
- Klucz obcy: **OrderID** → **Orders(OrderID)**
- Klucz obcy: **ProductID** → **Products(ProductID)**

3.11.3. Warunki integralności

- **NOT NULL** – wszystkie pola w tabeli są wymagane.
- CHECK: **CK_OrderDetails_UnitPrice** – cena jednostkowa produktu musi być nieujemna (**UnitPrice ≥ 0**).
- CHECK: **CK_OrderDetails_Quantity** – liczba sztuk produktu musi być dodatnia (**Quantity > 0**).

3.12. Tabela Inventory Products

```sql
CREATE TABLE dbo.InventoryProducts(
    ProductID INT NOT NULL UNIQUE
        CONSTRAINT FK_InventoryProducts_Products
        FOREIGN KEY REFERENCES dbo.Products(ProductID),
    Quantity INT NOT NULL
        CONSTRAINT CK_InventoryProducts_Quantity CHECK (Quantity >= 0),
    MinQuantity INT NOT NULL
        CONSTRAINT CK_InventoryProducts_MinQuantity CHECK (MinQuantity >= 0)
);
```

Rysunek 13: Struktura tabeli Inventory Products

Tabela **Inventory Products** przechowuje informacje o stanie na magazynie poszczególnych produktów.

3.12.1. Pola Tabeli

- **ProductID (INT)** - klucz obcy, identyfikator produktu. Pole wymagane i unikalne.
- **Quantity (INT)** - liczba sztuk produktu na magazynie. Pole wymagane.
- **MinQuantity (INT)** - minimalna dopuszczalna liczba sztuk produktu. Pole wymagane.

3.12.2. Klucze

- Klucz obcy: ProductID -> Products(ProductID)

3.12.3. Warunki integralności

- NOT NULL – wszystkie pola w tabeli są wymagane.
- CHECK: **CK_InventoryProducts_Quantity** – liczba sztuk produktu na magazynie musi być nieujemna.
- CHECK: **CK_InventoryProducts_MinQuantity** – minimalna dopuszczalna liczba sztuk produktu na magazynie musi być nieujemna.

4. Funkcje

4.1. Funkcja fn_AvgOrderCompletionTime

```
create function dbo.fn_AvgOrderCompletionTime ()
returns decimal(10,2) as
    begin
        declare @AvgTime decimal(10,2)
        select @AvgTime = avg(datediff(day, o.OrderDate, o.DeliveryDate))
        from Orders o
        return @AvgTime
    end
```

Rysunek 14: Definicja funkcji fn_AvgOrderCompletionTime

Funkcja **fn_AvgOrderCompletionTime** zwraca średni czas realizacji wszystkich zamówień, liczony w dniach.

4.1.1. Parametry

- brak

4.1.2. Zwracana wartość

- **DECIMAL(10,2)** – średni czas liczony w dniach, liczba zmiennoprzecinkowa.

4.2. Funkcja fn_EmployeeAccess

```
create function dbo.fn_EmployeeAccess(@EmployeeID int)
returns table with schemabinding
as
    return select 1 as allowed
    where exists(
        select 1
        from dbo.Employees e
        where e.EmployeeID = @EmployeeID
    )
```

Rysunek 15: Definicja funkcji fn_EmployeeAccess

Funckja **fn_EmployeeAccess** zwraca jeden wiersz z tabeli **Employees** należący do danego pracownika.

4.2.1. Paramtery

- **INT** - identyfikator pracownika

4.2.2. Zwracana wartość

- **Table** - tabela z jedym wierszem, należąca do danego pracownika.

4.3. Funkcja fn_LastOrderOfCustomer

```sql
create function dbo.fn_LastOrderOfCustomer (@CustomerID INT)
returns datetime as
    begin
        declare @LastOrderDate datetime
        select @LastOrderDate = max(o.OrderDate)
        from Orders o
        where o.CustomerID = @CustomerID
        return @LastOrderDate
    end;
```

Rysunek 16: Definicja funkcji fn_LastOrderOfCustomer

Funkcja **fn_LastOrderOfCustomer** zwraca datę ostatniego zamówienia złożonego przez danego klienta.

4.3.1. Parametry

- brak

4.3.2. Zwracana wartość

- **DATETIME** – data ostatniego zamówienia złożonego przez klienta.

4.4. Funkcja fn_MostFreqOrderedProduct

```sql
create function dbo.fn_MostFreqOrderedProduct ()
returns NVARCHAR(200) as
    begin
        declare @MostOrderedProduct NVARCHAR(200)
        select top 1 @MostOrderedProduct = p.ProductName
        from OrderDetails od
            inner join Products p on p.ProductID = od.ProductID
        group by p.ProductName
        order by count(*) desc
        return @MostOrderedProduct;
    end
```

Rysunek 17: Definicja funkcji fn_MostFreqOrderedProduct

Funkcja **fn_MostFreqOrderedProduct** zwraca nazwę produktu, który był najczęściej zamawiany.

4.4.1. Parametry

- brak

4.4.2. Zwracana wartość

- **NVARCHAR(200)** – nazwa najczęściej zamawianego produktu.

4.5. Funkcja fn_OrderTotal

```sql
CREATE OR ALTER FUNCTION dbo.fn_OrderTotal (@OrderID INT)
RETURNS DECIMAL(12,2)
AS
BEGIN
  DECLARE @Total DECIMAL(12,2);
  SELECT
    @Total = CAST(ISNULL(SUM(od.UnitPrice * od.Quantity), 0) AS DECIMAL(12, 2))
  FROM dbo.Orders o
  LEFT JOIN dbo.OrderDetails od ON od.OrderID = o.OrderID
  WHERE o.OrderID = @OrderID;

  RETURN ISNULL(@Total, 0);
END
```

Rysunek 18: Definicja funkcji fn_OrderTotal

Funkcja **fn_OrderTotal** oblicza łączną wartość zamówienia na podstawie pozycji zapisanych w tabeli **OrderDetails**. Kwota wyznaczana jest jako suma iloczynów ceny jednostkowej oraz ilości dla wszystkich produktów w danym zamówieniu.

4.5.1. Parametry

- **@OrderID (INT)** – identyfikator zamówienia, dla którego obliczana jest całkowita wartość.

4.5.2. Zwracana wartość

- **DECIMAL(12,2)** – całkowita kwota zamówienia. W przypadku braku pozycji lub nieistniejącego zamówienia zwracane jest **0.00**

4.6. Funkcja fn_OrderProductionTimeScalar

```sql
CREATE OR ALTER FUNCTION dbo.fn_OrderProductionTimeScalar (@OrderID INT)
RETURNS INT
AS
BEGIN
  DECLARE @t INT;
  SELECT @t = ISNULL(MAX(p.ProductionTime), 0) + 2
  FROM dbo.OrderDetails od
  JOIN dbo.Products p ON p.ProductID = od.ProductID
  WHERE od.OrderID = @OrderID;
  RETURN @t;
END
```

Rysunek 19: Definicja funkcji fn_OrderProductionTimeScalar

Funkcja **fn_OrderProductionTimeScalar** wyznacza przewidywany czas realizacji zamówienia na podstawie produktów znajdujących się w danym zamówieniu. Czas obliczany jest jako maksymalny czas produkcji spośród zamówionych produktów powiększony o stały bufor **2**.

4.6.0.1. Parametry

- `@OrderID` (INT) – identyfikator zamówienia, dla którego obliczany jest czas realizacji.

4.6.0.2. Zwracana wartość

- INT – przewidywany czas realizacji zamówienia. W przypadku braku pozycji w zamówieniu zwracane jest **0 + 2** (tj. 2).

4.7. Funkcja fn_RemainingOrderAmount

```sql
create function dbo.fn_RemainingOrderAmount (@OrderID INT)
returns decimal(10,2) as
    begin
        declare @Amount decimal(10,2)
        select @Amount = od.UnitPrice * od.Quantity + o.DeliveryPrice - isnull(sum(p.Amount), 0)
        from OrderDetails od
            inner join Payments p on od.OrderID = p.OrderID
            inner join Orders o on od.OrderID = o.OrderID
        where od.OrderID = @OrderID
        group by od.UnitPrice, od.Quantity, o.DeliveryPrice
        return @Amount
    end;
```

Rysunek 20: Definicja funkcji fn_RemainingOrderAmount

Funkcja **fn_RemainingOrderAmount** zwraca wartość pozostałą do zapłaty dla danego zamówienia.

4.7.1. Parametry

- **OrderID(INT)** - identyfikator zamówienia.

4.7.2. Zwracana wartość

- **DECIMAL(10,2)** – kwota pozostała do zapłaty dla danego zamówienia.

4.8. Funkcja fn_RevenueInRange

```sql
CREATE OR ALTER FUNCTION dbo.fn_RevenueInRange
(
  @From DATETIME,
  @To   DATETIME
)
RETURNS DECIMAL(12,2)
AS
BEGIN
  DECLARE @rev DECIMAL(12,2);

  SELECT @rev = CAST(SUM(ISNULL(t.ItemsTotal,0)) AS DECIMAL(12,2))
  FROM dbo.Orders o
  OUTER APPLY (
    SELECT SUM(od.UnitPrice * od.Quantity) AS ItemsTotal
    FROM dbo.OrderDetails od
    WHERE od.OrderID = o.OrderID
  ) t
  WHERE o.OrderDate >= @From
    AND o.OrderDate <  DATEADD(DAY, 1, @To);

  RETURN ISNULL(@rev, 0);
END
```

Rysunek 21: Definicja funkcji fn_RevenueInRange

Funkcja **fn_RevenueInRange** oblicza łączny przychód ze sprzedaży w zadanym przedziale dat na podstawie wartości pozycji zamówień (suma `UnitPrice * Quantity`).

4.8.0.1. Parametry

- `@From` (DATETIME) – początek przedziału czasu.
- `@To` (DATETIME) – koniec przedziału czasu (włącznie; w implementacji uwzględniany jest zakres do końca dnia `@To`).

4.8.0.2. Zwracana wartość

- DECIMAL(12,2) – łączny przychód w podanym zakresie dat. Jeśli brak zamówień w zakresie, funkcja zwraca **0.00**.

4.9. Funkcja fn_OrderCount_ByCustomerID

```sql
CREATE OR ALTER FUNCTION dbo.fn_OrderCount_ByCustomerID (@CustomerID INT)
RETURNS INT
AS
BEGIN
  DECLARE @cnt INT;
  SELECT @cnt = COUNT(*)
  FROM dbo.Orders
  WHERE CustomerID = @CustomerID;

  RETURN ISNULL(@cnt, 0);
END
```

Rysunek 22: Definicja funkcji fn_OrderCount_ByCustomerID

Funkcja **fn_OrderCount_ByCustomerID** zwraca liczbę zamówień złożonych przez wskazanego klienta na podstawie danych z tabeli **Orders**.

4.9.0.1. Parametry

- `@CustomerID` (INT) – identyfikator klienta.

4.9.0.2. Zwracana wartość

- INT – liczba zamówień danego klienta. Jeśli klient nie ma zamówień, funkcja zwraca **0**.

4.10. Funkcja fn_EmployeesWithMostOrders

```sql
CREATE FUNCTION dbo.fn_EmployeesWithMostOrders()
RETURNS TABLE
AS
RETURN (
    SELECT TOP 1 WITH TIES
        e.EmployeeID,
        e.Name,
        e.Address,
        e.PhoneNumber,
        e.isActive,
        COUNT(o.OrderID) as TotalOrdersHandled
    FROM dbo.Employees as e
    INNER JOIN dbo.Orders as o ON e.EmployeeID = o.EmployeeID
        AND o.Status in (3, 4)
    GROUP BY
        e.EmployeeID,
        e.Name,
        e.Address,
        e.PhoneNumber,
        e.isActive
    ORDER BY TotalOrdersHandled DESC, e.Name ASC
    );
```

Rysunek 23: Definicja funkcji fn_EmployeesWithMostOrders

Funkcja **fn_EmployeesWithMostOrders** identyfikuje pracowników, którzy obsłużyli największą liczbę zrealizowanych zamówień. Wynik uwzględnia tylko zamówienia o statusie zrealizowanym (3 lub 4) i zwraca rekordy pracowników z najwyższym wynikiem, stosując wiązania w przypadku identycznej liczby zleceń.

4.10.1. Parametry

- Brak parametrów wejściowych – funkcja analizuje dane dla wszystkich pracowników zarejestrowanych w systemie.

4.10.2. Zwracana wartość

- **TABLE** - tabela zawierająca dane najskuteczniejszych pracowników:

– **EmployeeID (INT)** – unikalny identyfikator pracownika.

– **Name (NVARCHAR(200))** – imię i nazwisko pracownika.

– **Address (NVARCHAR(200))** – adres zamieszkania pracownika.

– **PhoneNumber (NVARCHAR(20))** – numer telefonu kontaktowego.

– **isActive (BIT)** – informacja o statusie aktywności pracownika.

– **TotalOrdersHandled (INT)** – łączna liczba obsłużonych zamówień o statusie 3 lub 4.

4.11. Funkcja fn_GetAverageOrderValue

```sql
CREATE FUNCTION dbo.fn_GetAverageOrderValue()
RETURNS DECIMAL(10, 2)
AS
    BEGIN
        DECLARE @AvgValue DECIMAL(10, 2);

        SELECT @AvgValue = AVG(OrderValue)
        FROM (
            SELECT SUM(od.Quantity * od.UnitPrice) + o.DeliveryPrice as OrderValue
            FROM dbo.Orders as o
            JOIN dbo.OrderDetails as od ON o.OrderID = od.OrderID
            WHERE o.Status = 4
            GROUP BY o.OrderID, o.DeliveryPrice
             ) as OrdersSubquery;

        RETURN ISNULL(@AvgValue, 0);
    end;
```

Rysunek 24: Definicja funkcji fn_EmployeesWithMostOrders

Funkcja **fn_GetAverageOrderValue** oblicza średnią wartość wszystkich sfinalizowanych zamówień w systemie. Wartość

pojedynczego zamówienia wyliczana jest jako suma iloczynów ilości i ceny jednostkowej produktów powiększona o koszt dostawy, a następnie wyciągana jest średnia arytmetyczna ze wszystkich takich transakcji.

4.11.1. Parametry

- Brak parametrów wejściowych – funkcja analizuje wszystkie archiwalne dane sprzedażowe dostępne w bazie.

4.11.2. Zwracana wartość

- **DECIMAL(10, 2)** – średnia wartość zamówienia wyrażona w formacie dziesiętnym. W przypadku braku sfinalizowanych zamówień funkcja zwraca wartość 0.

4.12. Funkcja fn_GetProductRevenue

```
CREATE FUNCTION dbo.fn_GetProductRevenue (@ProductID INT)
RETURNS DECIMAL(10, 2)
AS
    BEGIN
        DECLARE @TotalRevenue DECIMAL(10, 2);
        SELECT @TotalRevenue = SUM(Quantity * UnitPrice)
        FROM dbo.OrderDetails as od
        JOIN dbo.Orders as o ON od.OrderID = o.OrderID
        WHERE od.ProductID = @ProductID
            AND o.Status = 4
        RETURN ISNULL(@TotalRevenue, 0);
    end;
```

Rysunek 25: Definicja funkcji fn_GetProductRevenue

Funkcja **fn_GetProductRevenue** oblicza całkowity przychód wygenerowany przez konkretny produkt na podstawie zrealizowanych zamówień. Sumuje ona iloczyn ilości sprzedanych sztuk oraz ich ceny jednostkowej, biorąc pod uwagę wyłącznie transakcje zakończone sukcesem.

4.12.1. Parametry

- **@ ProductID (INT)** – unikalny identyfikator produktu, dla którego ma zostać wyliczony całkowity przychód.

4.12.2. Zwracana wartość

- **DECIMAL(10, 2)** – łączna kwota przychodu uzyskanego ze sprzedaży danego produktu.
- W przypadku, gdy produkt nie został jeszcze sprzedany lub żadne zamówienie z jego udziałem nie zostało sfinalizowane, funkcja zwraca 0.

4.13. Funkcja fn_LeastBoughtProducts

```sql
CREATE FUNCTION dbo.fn_LeastBoughtProducts (@Count INT)
RETURNS TABLE
as
RETURN (
    SELECT TOP (@Count)
        p.ProductName,
        ISNULL(SUM(od.Quantity), 0) AS TotalQuantitySold
    FROM dbo.Products AS p
    LEFT JOIN (
        SELECT od.ProductID, od.Quantity
        FROM dbo.OrderDetails as od
        INNER JOIN dbo.Orders as o
            ON od.OrderID = o.OrderID
        WHERE o.Status = 4
    ) as od ON p.ProductID = od.ProductID
    GROUP BY p.ProductName
    ORDER BY TotalQuantitySold ASC, p.ProductName ASC

    );
```

Rysunek 26: Definicja funkcji fn_LeastBoughtProducts

Funkcja **fn_LeastBoughtProducts** generuje zestawienie produktów, które cieszą się najmniejszym zainteresowaniem wśród klientów. Zlicza ona łączną liczbę sprzedanych sztuk dla każdego produktu, biorąc pod uwagę wyłącznie sfinalizowane transakcje.

4.13.1. Parametry

- **@ Count (INT)** – określa liczbę produktów, które mają zostać zwrócone w rankingu (np. 5 najrzadziej kupowanych produktów).

4.13.2. Zwracana wartość

- **TABLE** – tabela zawierająca listę produktów o najniższej sprzedaży:

– **ProductName (NVARCHAR(200))** – nazwa handlowa produktu.

– **TotalQuantitySold (INT)** – łączna liczba sztuk sprzedanych w ramach sfinalizowanych zamówień.

5. Procedury

5.1. Procedura AddEmployee

```
CREATE PROCEDURE dbo.AddEmployee
    @Name NVARCHAR(200),
    @Address NVARCHAR(200),
    @PhoneNumber NVARCHAR(20),
    @isActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Employees (Name, Address, PhoneNumber, isActive)
    OUTPUT inserted.EmployeeID
    VALUES (@Name, @Address, @PhoneNumber, @isActive);
END
GO
```

Rysunek 27: Definicja procedury AddEmployee

Procedura **AddEmployee** dodaje nowego pracownika do tabeli **Employees**.

5.2. Procedura AddSupplier

```
CREATE PROCEDURE dbo.AddSupplier
    @CompanyName NVARCHAR(200),
    @Address NVARCHAR(200),
    @City NVARCHAR(200),
    @Country NVARCHAR(200),
    @PhoneNumber NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Suppliers (CompanyName, Address, City, Country, PhoneNumber)
    OUTPUT inserted.SupplierID
    VALUES (@CompanyName, @Address, @City, @Country, @PhoneNumber);
end
GO
```

Rysunek 28: Definicja procedury AddSupplier

Procedura **AddSupplier** dodaje nowego dostarczyciela zamówień do tabeli **Suppliers**.

5.3. Procedura AddOrderDetails

```sql
CREATE PROCEDURE dbo.AddOrderDetails
    @OrderID INT,
    @ProductID INT,
    @UnitPrice DECIMAL(10,2),
    @Quantity INT
AS
    BEGIN
        SET NOCOUNT ON;
        INSERT INTO dbo.OrderDetails (OrderID, ProductID, UnitPrice, Quantity)
        VALUES(@OrderID, @ProductID, @UnitPrice, @Quantity);
    end
go
```

Rysunek 29: Definicja procedury AddOrderDetails

Procedura **AddOrderDetails** dodaje rekord szczegółów zamówienia do tabeli **OrderDetails**. Określa ona produkt, jego cenę jednostkową oraz ilość. Dane te są później wykorzystywane do obliczenia całkowitej wartości zamówienia oraz do automatycznego zmniejszenia stanu magazynowego po pełnym opłaceniu zamówienia.

5.4. Procedura AddInventoryProduct

```sql
CREATE PROCEDURE dbo.AddInventoryProduct
    @ProductID INT,
    @Quantity INT
AS
    BEGIN
        SET NOCOUNT ON;
        INSERT INTO dbo.InventoryProducts (ProductID, Quantity)
        VALUES (@ProductID, @Quantity);
    end
go
```

Rysunek 30: Definicja procedury AddOrderDetails

Procedura **AddInventoryProduct** dodaje rekord szczegółów stanu magazynowego produktu do tabeli **IventoryProducts**.

5.5. Procedura AddOrder

```sql
CREATE OR ALTER PROCEDURE dbo.AddOrder
    @CustomerID INT,
    @EmployeeID INT,
    @ShipperID INT,
    @OrderDate DATETIME,
    @DeliveryDate DATETIME,
    @Address NVARCHAR(200),
    @Country NVARCHAR(100),
    @City NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Orders
        (CustomerID, EmployeeID, ShippersID, OrderDate, DeliveryDate, Address, Country, City)
    VALUES
        (@CustomerID, @EmployeeID, @ShipperID, @OrderDate, @DeliveryDate, @Address, @Country, @City);

    SELECT CAST(SCOPE_IDENTITY() AS INT) AS OrderID;
END
GO
```

Rysunek 31: Definicja procedury AddOrder

Procedura **AddOrder** dodaje nowe zamówienie do tabeli **Orders** na podstawie przekazanych danych klienta, pracownika, przewoźnika oraz informacji adresowych. Nowe zamówienie otrzymuje domyślnie status **2 (nieopłacone)**. Po dodaniu rekordu procedura zwraca identyfikator utworzonego zamówienia (**OrderID**), który jest następnie wykorzystywany do dodawania pozycji zamówienia oraz rejestrowania płatności.

5.6. Procedura ChangeOrderData

```sql
CREATE   PROCEDURE dbo.ChangeOrderData
    @OrderID      INT,
    @DeliveryDate DATETIME = NULL,
    @Address      NVARCHAR(200) = NULL,
    @Country      NVARCHAR(100) = NULL,
    @City         NVARCHAR(100) = NULL
AS
    BEGIN
        SET NOCOUNT ON;

        UPDATE dbo.Orders
        SET
            DeliveryDate = COALESCE(@DeliveryDate, DeliveryDate),
            Address = COALESCE(@Address, Address),
            Country = COALESCE(@Country, Country),
            City = COALESCE(@City, City)
        WHERE OrderID = @OrderID;
    END;
```

Rysunek 32: Definicja procedury ChangeOrderData

Procedura **ChangeOrderData** umożliwia aktualizację wybranych danych zamówienia: daty dostawy oraz danych adresowych. Parametry opcjonalne pozwalają zmienić tylko te pola, które zostały przekazane (pozostałe pozostają bez zmian).

5.7. Procedura AssignShippers

```sql
CREATE   PROCEDURE dbo.AssignShippers
    @OrderID    INT,
    @ShippersID INT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Orders
    SET ShippersID = @ShippersID
    WHERE OrderID = @OrderID;

END;
```

Rysunek 33: Definicja procedury AssignShippers

Procedura **AssignShippers** przypisuje przewoźnika do wskazanego zamówienia poprzez aktualizację pola **ShippersID** w tabeli **Orders**.

5.8. Procedura AddCustomer

```sql
CREATE   PROCEDURE dbo.AddCustomer
  @Name         NVARCHAR(100),
  @Address      NVARCHAR(200),
  @PhoneNumber  NVARCHAR(30)
AS
BEGIN
  SET NOCOUNT ON;

  INSERT INTO dbo.Customers (Name, Address, PhoneNumber)
  OUTPUT INSERTED.CustomerID
  VALUES (@Name, @Address, @PhoneNumber);
END
```

Rysunek 34: Definicja procedury AddCustomer

Procedura **AddCustomer** dodaje nowego klienta do tabeli **Customers** na podstawie przekazanych danych (nazwa, adres, numer telefonu).

5.9. Procedura ChangeCustomerData

```sql
CREATE   PROCEDURE dbo.ChangeCustomerData
  @CustomerID   INT,
  @Name         NVARCHAR(100),
  @Address      NVARCHAR(200)
AS
BEGIN
  SET NOCOUNT ON;

  UPDATE dbo.Customers
     SET Name = @Name,
         Address = @Address
   WHERE CustomerID = @CustomerID;

END
```

Rysunek 35: Definicja procedury ChangeCustomerData

Procedura **ChangeCustomerData** aktualizuje podstawowe dane klienta w tabeli **Customers** (nazwa oraz adres) dla wskazanego **CustomerID**.

5.10. Procedura ChangeCustomerContact

```sql
CREATE   PROCEDURE dbo.ChangeCustomerContact
  @CustomerID   INT,
  @PhoneNumber  NVARCHAR(30)
AS
BEGIN
  SET NOCOUNT ON;

  UPDATE dbo.Customers
     SET PhoneNumber = @PhoneNumber
   WHERE CustomerID = @CustomerID;

END
```

Rysunek 36: Definicja procedury ChangeCustomerContact

Procedura **ChangeCustomerContact** aktualizuje numer telefonu klienta w tabeli **Customers** dla wskazanego **CustomerID**.

5.11. Procedura ChangeEmployeeData

```sql
CREATE PROCEDURE dbo.ChangeEmployeeData
    @EmployeeID INT,
    @Name NVARCHAR(200),
    @Address NVARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Employees
    SET
        Name = @Name,
        Adddress = @Address
    WHERE EmployeeID = @EmployeeID;
end
go
```

Rysunek 37: Definicja procedury ChangeEmployeeData

Procedura **ChangeEmployeeData** aktualizuje numer telefonu i adres pracownika w tabeli **Employees** dla wskazanego **EmployeeID**.

5.12. Procedura ChangeEmployeeContact

```sql
CREATE PROCEDURE dbo.ChangeEmployeeContact
    @EmployeeID INT,
    @PhoneNumber NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Employees
    SET PhoneNumber = @PhoneNumber
    WHERE EmployeeID = @EmployeeID;
end
go
```

Rysunek 38: Definicja procedury ChangeEmployeeContact

Procedura **ChangeEmployeeContact** aktualizuje numer telefonu pracownika w tabeli **Employees** dla wskazanego **EmployeeID**.

5.13. Procedura ChangeIsActiveEmployee

```sql
CREATE PROCEDURE dbo.ChangeIsActiveEmployee
    @EmployeeID INT,
    @isActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Employees
    SET isActive = @isActive
    WHERE EmployeeID = @EmployeeID;
end
go
```

Rysunek 39: Definicja procedury ChangeIsActiveEmployee

Procedura **ChangeIsActiveEmployee** aktualizuje status pracownika w tabeli **Employees** dla wskazanego **EmployeeID**.

5.14. Procedura ChangeSupplierData

```sql
CREATE PROCEDURE dbo.ChangeSupplierData
    @SupplierID INT,
    @CompanyName NVARCHAR(200),
    @Address NVARCHAR(200),
    @City NVARCHAR(200),
    @Country NVARCHAR(200),
    @PhoneNumber NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Suppliers
    SET
        CompanyName = @CompanyName,
        Address = @Address,
        City = @City,
        Country = @Country,
        PhoneNumber = @PhoneNumber
    WHERE SupplierID = @SupplierID;
end
go
```

Rysunek 40: Definicja procedury ChangeSupplierData

Procedura **ChangeSupplierData** aktualizuje dane dostawcy wysyłek w tabeli **Suppliers** dla wskazanego **SupplierID**.

5.15. Procedura ChangeSupplierContact

```sql
CREATE PROCEDURE dbo.ChangeSupplierContact
    @SupplierID INT,
    @PhoneNumber NVARCHAR(20)
AS
    BEGIN
        SET NOCOUNT ON;
        UPDATE dbo.Suppliers
        SET PhoneNumber = @PhoneNumber
        WHERE SupplierID = @SupplierID;
    end
go
```

Rysunek 41: Definicja procedury ChangeSupplierData

Procedura **ChangeSupplierData** aktualizuje dane kontaktowe dostawcy wysyłek w tabeli **Suppliers** dla wskazanego **SupplierID**.

5.16. Procedura AddProduct

```sql
CREATE   PROCEDURE dbo.AddProduct
  @ProductName     NVARCHAR(200),
  @Price           DECIMAL(10,2),
  @ProductionTime  INT
AS
BEGIN
  SET NOCOUNT ON;

  INSERT INTO dbo.Products (ProductName, Price, ProductionTime)
  VALUES (@ProductName, @Price, @ProductionTime);

END
```

Rysunek 42: Definicja procedury AddProduct

Procedura **AddProduct** dodaje nowy produkt do tabeli **Products** na podstawie przekazanych parametrów: nazwy produktu, ceny oraz czasu produkcji.

5.17. Procedura ChangeProductData

```sql
CREATE   PROCEDURE dbo.ChangeProductData
  @ProductID       INT,
  @ProductName     NVARCHAR(200),
  @Price           DECIMAL(10,2),
  @ProductionTime  INT
AS
BEGIN
  SET NOCOUNT ON;

  UPDATE dbo.Products
     SET ProductName = @ProductName,
         Price = @Price,
         ProductionTime = @ProductionTime
   WHERE ProductID = @ProductID;

END
```

Rysunek 43: Definicja procedury ChangeProductData

Procedura **ChangeProductData** aktualizuje dane produktu w tabeli **Products** (nazwa, cena oraz czas produkcji) dla wskazanego **ProductID**.

5.18. Procedura AddInventoryPart

```sql
CREATE PROCEDURE dbo.AddInventoryPart
    @PartID INT,
    @Quantity INT,
    @MinQuantity INT

    AS
    BEGIN
        SET NOCOUNT ON;

        INSERT INTO dbo.InventoryParts(PartID, Quantity, MinQuantity)
        VALUES (@PartID, @Quantity, @MinQuantity);
    end
go
```

Rysunek 44: Definicja procedury AddInventoryPart

Procedura **AddInventoryPart** służy do inicjowania stanów magazynowych dla nowych komponentów w systemie. Pozwala ona na wprowadzenie nowej części do ewidencji magazynowej wraz z określeniem jej początkowej ilości oraz progu alarmowego (ilości minimalnej), co jest niezbędne dla poprawnego funkcjonowania procesów zaopatrzenia.

5.19. Procedura AddPart

```sql
CREATE PROCEDURE dbo.AddPart
    @SupplierID INT,
    @PartName NVARCHAR(200),
    @Cost DECIMAL(10, 2)

AS
    BEGIN
        SET NOCOUNT ON;

        INSERT INTO dbo.Parts(SupplierID, PartName, Cost)
        OUTPUT inserted.PartID
        VALUES (@SupplierID, @PartName, @Cost);
    end
GO
```

Rysunek 45: Definicja procedury AddPart

Procedura **AddPart** służy do rejestracji nowych komponentów w katalogu firmy. Pozwala ona na wprowadzenie nazwy części, przypisanie jej do konkretnego dostawcy oraz określenie kosztu zakupu, zwracając jednocześnie nowo wygenerowany identyfikator zasobu.

5.20. Procedura AddPayment

```
CREATE OR ALTER PROCEDURE dbo.AddPayment
    @OrderID INT,
    @Amount DECIMAL(10,2),
    @STATUS INT,
    @Method NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @out TABLE (PaymentID INT);

    INSERT INTO dbo.Payments (OrderID, PaymentDate, Amount, STATUS, Method)
    OUTPUT inserted.PaymentID INTO @out(PaymentID)
    VALUES (@OrderID, GETDATE(), @Amount, @STATUS, @Method);

    SELECT PaymentID FROM @out;
END
GO
```

Rysunek 46: Definicja procedury AddPayment

Procedura **AddPayment** służy do rejestrowania wpłat dokonywanych przez klientów dla konkretnych zamówień. Kwota płatności przekazywana jest jako wartość typu **DECIMAL**. Procedura zapisuje datę wykonania operacji oraz zwraca identyfikator nowej płatności.

5.21. Procedura AddShipper

```sql
CREATE PROCEDURE dbo.AddShipper
    @CompanyName NVARCHAR(200),
    @PhoneNumber NVARCHAR(20),
    @isActive BIT

AS
    BEGIN
        SET NOCOUNT ON;

        INSERT INTO dbo.Shippers(CompanyName, PhoneNumber, isActive)
        OUTPUT inserted.ShipperID
        VALUES (@CompanyName, @PhoneNumber, @isActive);
    end
go
```

Rysunek 47: Definicja procedury AddShipper

Procedura **AddShipper** umożliwia dodawanie nowych firm do bazy danych partnerów logistycznych. Pozwala na zdefiniowanie nazwy firmy, numeru kontaktowego oraz początkowego statusu aktywności, co jest niezbędne do zlecania transportu gotowych produktów.

5.22. Prodcedura ChangeIsActiveShipper

```sql
CREATE PROCEDURE dbo.ChangeIsActiveShipper
    @ShipperID INT,
    @isActive BIT
    AS
    BEGIN
        SET NOCOUNT ON;

        UPDATE dbo.Shippers
        SET
            isActive = @isActive
        WHERE ShipperID = @ShipperID
    end
go
```

Rysunek 48: Definicja procedury ChangeIsActiveShipper

Procedura **ChangeIsActiveShipper** służy do zarządzania dostępnością firm w systemie logistycznym. Umożliwia ona szybką zmianę statusu aktywności wybranego przewoźnika.

5.23. Procedura ChangePartData

```sql
CREATE PROCEDURE dbo.ChangePartData
    @PartID INT,
    @PartName NVARCHAR(200),
    @Cost DECIMAL(10, 2)
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Parts
    SET
        PartName = @PartName,
        Cost = @Cost
    WHERE PartID = @PartID;
end
go
```

Rysunek 49: Definicja procedury ChangePartData

Procedura **ChangePartData** umożliwia aktualizację kluczowych informacji dotyczących komponentów wykorzystywanych w produkcji. Służy do modyfikacji nazwy technicznej oraz kosztu zakupu części.

5.24. Procedura ChangeShipperContact

```sql
CREATE PROCEDURE dbo.ChangeShipperContact
    @ShipperID INT,
    @PhoneNumber NVARCHAR(20)

    AS
    BEGIN
        SET NOCOUNT ON;

        UPDATE dbo.Shippers
        SET
            PhoneNumber = @PhoneNumber
        WHERE ShipperID = @ShipperID
    end
go
```

Rysunek 50: Definicja procedury ChangeShipperContact

Procedura ChangeShipperContact umożliwia aktualizację danych kontaktowych. Pozwala na szybką zmianę numeru telefonu przypisanego do konkretnego przewoźnika.

5.25. Procedura ChangeShipperData

```sql
CREATE PROCEDURE dbo.ChangeShipperData
    @ShipperID INT,
    @CompanyName NVARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Shippers
    SET
        CompanyName = @CompanyName
    WHERE ShipperID = @ShipperID;
end
go
```

Rysunek 51: Definicja procedury ChangeShipperData

Procedura **ChangeShipperData** służy do aktualizacji nazwy firmy przewoźnika zarejestrowanej w systemie.

6. Triggery

6.1. Trigger trg_BlockInactiveEmployees

```sql
create or alter trigger dbo.trg_BlockInactiveEmployees
on dbo.Orders
    after insert, update
    as
    begin
        set nocount on;

        declare @InactiveEmployeeID int;
        declare @ErrorMsg NVARCHAR(100);

        select top 1 @InactiveEmployeeID = e.EmployeeID
        from inserted i
        inner join Employees e
            on i.EmployeeID = e.EmployeeID
        where e.isActive = 0;

        if @InactiveEmployeeID is not null
        begin
            set @ErrorMsg = concat(
                            'Nie można przypisać nieaktywnego pracownika (ID: ', cast(@InactiveEmployeeID as NVARCHAR(10)), ') do zamówienia'
                            );
            throw 50001, @ErrorMsg, 1;
        end
    end
```

Rysunek 52: Definicja triggera trg_BlockInactiveEmployees

Trigger **trg_BlockInactiveEmployees** uruchamia się po próbie utworzenia lub zmian rekordu w tabeli **Orders**. Jeżeli pracownik przypisany po zamówienia ma **STATUS** nieaktywny, to trigger wyrzuca błąd (`THROW`).

6.2. Trigger trg_InventoryParts_AutoAddParts

```sql
create or alter trigger dbo.trg_InventoryParts_AutoAddParts
    on dbo.InventoryParts
    after update
    as
    begin
        set nocount on;
        if not update(Quantity)
            return;

        update ip
        set ip.Quantity = ip.Quantity + (ip.MinQuantity * 3)
        from InventoryParts ip
        inner join inserted as i on ip.PartID = i.PartID
        inner join deleted as d on ip.PartID = d.PartID
        where i.Quantity < i.MinQuantity
            and i.Quantity < d.Quantity
    end;
go
```

Rysunek 53: Definicja triggera trg_InventoryParts_AutoAddParts

Trigger **trg_InventoryParts_AutoAddParts** uruchamia się po aktualizacji kolumny **Quantity** w tabeli **InventoryParts**. Dla części, której stan magazynu spadł poniżej minimalnej wartości, automatycznie jest zamawiana jest odgórnie trzykrotność wartości **MinQuantity** tej części.

6.3. Trigger trg_Orders_PredictOrderExecution

```sql
CREATE OR ALTER TRIGGER dbo.trg_Orders_PredictOrderExecution
ON dbo.Orders
AFTER INSERT, UPDATE
AS
BEGIN
  SET NOCOUNT ON;

  ;WITH changed AS (
    SELECT i.OrderID, i.Status, d.Status AS OldStatus
    FROM inserted i
    LEFT JOIN deleted d ON d.OrderID = i.OrderID
    WHERE i.Status = 3
      AND (d.OrderID IS NULL OR ISNULL(d.Status, -1) <> 3)
  ),
  agg AS (
    SELECT
      c.OrderID,
      COALESCE(MAX(p.ProductionTime), 0) + 2 AS ExecDays
    FROM changed c
    LEFT JOIN dbo.OrderDetails oi ON oi.OrderID = c.OrderID
    LEFT JOIN dbo.Products p ON p.ProductID = oi.ProductID
    GROUP BY c.OrderID
  ),
  vals AS (
    SELECT
      a.OrderID,
      DATEADD(DAY, a.ExecDays, GETDATE()) AS PredictedOrderDelivery
    FROM agg a
  )
  MERGE dbo.OrderExecution AS tgt
  USING vals AS src
     ON tgt.OrderID = src.OrderID
  WHEN MATCHED THEN
    UPDATE SET PredictedOrderDelivery = src.PredictedOrderDelivery
  WHEN NOT MATCHED THEN
    INSERT (OrderID, PredictedOrderDelivery)
    VALUES (src.OrderID, src.PredictedOrderDelivery);
END;
```

Rysunek 54: Definicja triggera trg_Orders_PredictOrderExecution

Trigger **trg_Orders_PredictOrderExecution** uruchamia się po dodaniu lub aktualizacji rekordu w tabeli **Orders**. Działa wyłącznie dla zamówień, które otrzymały status **3** (co oznacza,

że zamówienie jest opłacone i niezrealizowane). Na podstawie pozycji zamówienia (**OrderDetails**) wyznacza przewidywaną datę realizacji jako aktualna data + (maksymalny czas produkcji z zamówionych produktów + 2 dni buforu) i zapisuje wynik w tabeli **OrderExecution** (aktualizuje wpis istniejący lub tworzy nowy).

6.4. Trigger trg_Payments_UpdatePaymentStatus

```sql
CREATE OR ALTER TRIGGER dbo.trg_Payments_UpdatePaymentStatus
ON dbo.Payments
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH changed_orders AS (
        SELECT OrderID FROM inserted
        UNION
        SELECT OrderID FROM deleted
    ),
    order_totals AS (
        SELECT
            o.OrderID,
            CAST(
                ROUND(
                    COALESCE(SUM(od.UnitPrice * od.Quantity), 0)
                    + COALESCE(o.DeliveryPrice, 0),
                2) * 100
            AS bigint) AS TotalCents
        FROM dbo.Orders o
        LEFT JOIN dbo.OrderDetails od ON od.OrderID = o.OrderID
        WHERE o.OrderID IN (SELECT OrderID FROM changed_orders)
        GROUP BY o.OrderID, o.DeliveryPrice
    ),
    paid_totals AS (
        SELECT
            p.OrderID,
            CAST(ROUND(COALESCE(SUM(p.Amount), 0), 2) * 100 AS bigint) AS PaidCents
        FROM dbo.Payments p
        WHERE p.OrderID IN (SELECT OrderID FROM changed_orders)
        GROUP BY p.OrderID
    ),
    combined AS (
        SELECT
            ot.OrderID,
            ot.TotalCents,
            ISNULL(pt.PaidCents, 0) AS PaidCents
        FROM order_totals ot
        LEFT JOIN paid_totals pt ON pt.OrderID = ot.OrderID
    )

    UPDATE p
    SET p.STATUS =
        CASE
            WHEN c.TotalCents > 0 AND c.PaidCents >= c.TotalCents THEN 1
            ELSE 0
        END
    FROM dbo.Payments p
    JOIN combined c ON c.OrderID = p.OrderID
    WHERE p.OrderID IN (SELECT OrderID FROM changed_orders);
END;
GO
```

Rysunek 55: Definicja triggera trg_Payments_UpdatePaymentStatus

Trigger **trg_Payments_UpdatePaymentStatus** uruchamia się po dodaniu, zmianie lub usunięciu płatności w tabeli **Payments**. Dla zamówień powiązanych ze zmienionymi płatnościami oblicza całkowitą wartość zamówienia (suma pozycji **OrderDetails** wraz z kosztem dostawy **DeliveryPrice**) oraz sumę wpłat w tabeli **Payments**. Następnie aktualizuje pole **Status** w tabeli **Payments**: jeśli suma wpłat jest większa lub równa wartości zamówienia (i wartość zamówienia > 0), status ustawiany jest na **1**, w przeciwnym razie na **0**.

6.5. Trigger trg_Orders_AddOrderDate

```sql
CREATE OR ALTER TRIGGER dbo.trg_AddOrderDate
ON dbo.Payments
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT UPDATE(Status)
        RETURN;

    UPDATE o
    SET o.OrderDate = GETDATE()
    FROM dbo.Orders o
    JOIN inserted i ON i.OrderID = o.OrderID
    JOIN deleted  d ON d.PaymentID = i.PaymentID
    WHERE i.Status = 1
      AND d.Status <> 1;
END
GO
```

Rysunek 56: Definicja triggera trg_Orders_AddOrderDate

Trigger **trg_Orders_AddOrderDate** uruchamia się po aktualizacji kolumny **Status** w tabeli **Payments**. Jeżeli status płatności dla danego rekordu zmieni się na **1** (płatność w pełni

opłacona), trigger ustawia w tabeli **Orders** pole **OrderDate** na aktualną datę i czas (**GETDATE()**) dla zamówienia powiązanego przez **OrderID**.

6.6. Trigger trg_Orders_InventoryDecrement

```sql
CREATE OR ALTER TRIGGER dbo.trg_Orders_InventoryDecrement
ON dbo.Orders
AFTER INSERT, UPDATE
AS
BEGIN
  SET NOCOUNT ON;

  DECLARE @changed TABLE (OrderID INT PRIMARY KEY);

  INSERT INTO @changed(OrderID)
  SELECT i.OrderID
  FROM inserted i
  LEFT JOIN deleted d ON d.OrderID = i.OrderID
  WHERE i.Status = 3
    AND (d.OrderID IS NULL OR d.Status <> 3);

  IF NOT EXISTS (SELECT 1 FROM @changed)
    RETURN;

  DECLARE @req TABLE (ProductID INT PRIMARY KEY, ReqQty INT NOT NULL);

  INSERT INTO @req(ProductID, ReqQty)
  SELECT od.ProductID,
         SUM(od.Quantity) AS ReqQty
  FROM dbo.OrderDetails od
  JOIN @changed c ON c.OrderID = od.OrderID
  GROUP BY od.ProductID;

  IF EXISTS (
      SELECT 1
      FROM @req r
      LEFT JOIN dbo.InventoryProducts ip ON ip.ProductID = r.ProductID
      WHERE ip.ProductID IS NULL
  )
  BEGIN
    THROW 51001, N'Brak pozycji w magazynie (InventoryProducts) dla co najmniej jednego produktu z zamówienia.', 1;
  END;

  IF EXISTS (
      SELECT 1
      FROM @req r
      JOIN dbo.InventoryProducts ip ON ip.ProductID = r.ProductID
      WHERE ip.Quantity < r.ReqQty
  )
  BEGIN
    THROW 51002, N'Niewystarczający stan magazynowy (InventoryProducts) dla co najmniej jednego produktu z zamówienia.', 1;
  END;

  UPDATE ip
     SET ip.Quantity = ip.Quantity - r.ReqQty
  FROM dbo.InventoryProducts ip
  JOIN @req r ON r.ProductID = ip.ProductID;
END
GO
```

Rysunek 57: Definicja triggera trg_Orders_InventoryDecrement

Trigger **trg_Orders_InventoryDecrement** uruchamia się po dodaniu lub aktualizacji rekordu w tabeli **Orders**. Działa dla zamówień, które zostały ustawione na status **3** (co oznacza, że zamówienie jest opłacone i niezrealizowane). Trigger wylicza zapotrzebowanie na produkty na podstawie pozycji zamówienia

(**OrderDetails**) i sprawdza, czy istnieją odpowiadające wpisy w **InventoryProducts** oraz czy stan magazynu jest wystarczający. W przypadku braków zgłaszany jest błąd (`THROW`). Jeśli warunki są spełnione, trigger zmniejsza stan magazynowy w **InventoryProducts** o wymaganą ilość.

6.7. Trigger trg_InventoryProducts_AutoAddStock

```sql
CREATE OR ALTER TRIGGER dbo.trg_InventoryProducts_AutoAddStock
    on dbo.InventoryProducts
    AFTER UPDATE
    AS
    BEGIN
        SET NOCOUNT ON;

        IF NOT UPDATE(Quantity)
            RETURN;

        UPDATE ip
        SET ip.Quantity = ip.Quantity + 1 + (ip.MinQuantity * 3)
        FROM InventoryProducts as ip
        INNER JOIN inserted as i ON ip.ProductID = i.ProductID
        INNER JOIN deleted as d ON ip.ProductID = d.ProductID
        WHERE i.Quantity < i.MinQuantity
        AND i.Quantity < d.Quantity;
    end;
go
```

Rysunek 58: Definicja triggera trg_InventoryProducts_AutoAddStock

Trigger **trg_InventoryProducts_AutoAddStock** pełni funkcję automatycznego systemu uzupełniania zapasów produktów gotowych. Monitoruje on zmiany stanów magazynowych i w momencie wykrycia krytycznego spadku liczby produktów (poniżej poziomu minimalnego), automatycznie symuluje „domówienie” towaru, natychmiastowo zwiększając jego dostępną ilość.

6.8. Trigger trg_OrdersBlockInactiveShippers

```sql
CREATE OR ALTER TRIGGER dbo.trg_OrdersBlockInactiveShippers
ON dbo.Orders
    AFTER INSERT, UPDATE
    AS
    BEGIN
        SET NOCOUNT ON;

        DECLARE @InactiveShipperID INT;
        DECLARE @ErrorMessage NVARCHAR(100);

        SELECT TOP 1 @InactiveShipperID = s.ShipperID
        FROM inserted as i
        JOIN Shippers as s
        ON i.ShippersID = s.ShipperID
        WHERE s.isActive = 0 AND i.ShippersID IS NOT NULL;

        IF @InactiveShipperID IS NOT NULL
        BEGIN
            SET @ErrorMessage = CONCAT('Nie można przypisać nieaktywnego dostawcy (ID: '
                                       CAST(@InactiveShipperID AS NVARCHAR(10)),
                                       ') do zamówienia.');
            THROW 50001, @ErrorMessage, 1;
        end
    end;
GO
```

Rysunek 59: Definicja triggera ttrg_OrdersBlockInactiveShippers

Trigger **trg_OrdersBlockInactiveShippers** stanowi mechanizm kontroli integralności biznesowej w procesie logistycznym. Jego zadaniem jest uniemożliwienie przypisania do zamówienia firmy, która została oznaczona w systemie jako nieaktywna. Zapewnia to, że towary będą wysyłane wyłącznie przez dostępnych przewoźników.

6.9. Trigger trg_Payments_OrderStatus2_To3_OnFullyPaid

```sql
CREATE OR ALTER TRIGGER dbo.trg_Payments_OrderStatus2_To3_OnFullyPaid
ON dbo.Payments
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH changed_orders AS (
        SELECT OrderID FROM inserted
        UNION
        SELECT OrderID FROM deleted
    ),
    totals AS (
        SELECT
            o.OrderID,
            o.Status AS OrderStatus,
            CAST(ROUND(COALESCE(SUM(od.UnitPrice*od.Quantity),0) + COALESCE(o.DeliveryPrice,0), 2) * 100 AS bigint) AS TotalCents
        FROM dbo.Orders o
        LEFT JOIN dbo.OrderDetails od ON od.OrderID = o.OrderID
        WHERE o.OrderID IN (SELECT OrderID FROM changed_orders)
        GROUP BY o.OrderID, o.Status, o.DeliveryPrice
    ),
    paid AS (
        SELECT
            p.OrderID,
            CAST(ROUND(COALESCE(SUM(p.Amount),0), 2) * 100 AS bigint) AS PaidCents
        FROM dbo.Payments p
        WHERE p.OrderID IN (SELECT OrderID FROM changed_orders)
        GROUP BY p.OrderID
    )
    UPDATE o
    SET o.Status = 3
    FROM dbo.Orders o
    JOIN totals t ON t.OrderID = o.OrderID
    JOIN paid   p ON p.OrderID = o.OrderID
    WHERE t.OrderStatus = 2
      AND t.TotalCents > 0
      AND p.PaidCents >= t.TotalCents;
END;
GO
```

Rysunek 60: Definicja triggera trg_Payments_OrderStatus2_To3_OnFullyPaid

Trigger **trg_Payments_OrderStatus2_To3_OnFullyPaid** uruchamia się po dodaniu, zmianie lub usunięciu płatności w tabeli **Payments**. Dla zamówień powiązanych ze zmienionymi płatnościami oblicza całkowitą wartość zamówienia (suma pozycji **OrderDetails** wraz z kosztem dostawy **DeliveryPrice**) oraz sumę wpłat. Jeżeli zamówienie ma aktualnie status **2** (nieopłacone) oraz suma wpłat jest większa lub równa wartości zamówienia (i wartość zamówienia > 0), trigger aktualizuje status zamówienia w tabeli **Orders** na **3** (opłacone i niezrealizowane).

7. Widoki

7.1. Widok v_ActiveEmployeesOrders

```sql
CREATE OR ALTER VIEW dbo.v_ActiveEmployeesOrders AS
SELECT
  e.EmployeeID,
  e.Name           AS EmployeeName,
  COUNT(o.OrderID) AS ActiveOrdersCount
FROM dbo.Employees e
LEFT JOIN dbo.Orders o
  ON o.EmployeeID = e.EmployeeID
 AND o.Status = 3
WHERE e.IsActive = 1
GROUP BY e.EmployeeID, e.Name;
GO
```

Rysunek 61: Definicja widoku v_ActiveEmployeesOrders

Widok **v_ActiveEmployeesOrders** przedstawia listę aktywnych pracowników wraz z liczbą przypisanych do nich zamówień o statusie **3**(gdy zamówienie jest opłacone i niezreazliowane).Widok uwzględnia również aktywnych pracowników, którzy aktualnie nie mają żadnych zamówień.

7.2. Widok v_CustomerRemainingAmounts

```sql
create or alter view dbo.v_CustomersRemainingAmounts as
    select c.Name,
           c.Address,
           c.PhoneNumber,
           sum(dbo.fn_RemainingOrderAmount(o.OrderID)) as RemainingAmountsSum
    from Customers c
        inner join Orders o on o.CustomerID = c.CustomerID
    group by
        o.OrderID, c.Name, c.Address, c.PhoneNumber
    having sum(dbo.fn_RemainingOrderAmount(o.OrderID)) > 0;
go
```

Rysunek 62: Definicja widoku v_CustomerRemainingAmounts

Widok **v_CustomerRemainingAmounts** przedstawia listę klientów wraz z sumą zaległych opłat ze wszystkich zamówień. Klienci bez długu są pomijani.

7.3. Widok v_InventoryProductsStockInfo

```sql
create or alter view dbo.v_InventoryProductsStockInfo as
    select p.ProductName,
           case
                when ip.Quantity > 3 * ip.MinQuantity then 'Wysoki stan'
                when ip.Quantity > 2 * ip.MinQuantity then N'Średni stan'
                else 'Niski stan'
            end
            as Stan_Magazynu
    from InventoryProducts ip
        inner join Products p on p.ProductID = ip.ProductID
```

Rysunek 63: Definicja widoku v_InventoryProductsStockInfo

Widok **v_InventoryProductsStockInfo** przedstawia listę produktów wraz z ich stanem magazynowym opisanym w stosunku do stanu minimalnego. **Stan wysoki** oznacza trzykrotność minimum, **stan średni** dwukrotność, a **stan niski** - stan nieznacznie przekraczający minimum.

7.4. Widok v_OrderBasic

```sql
CREATE OR ALTER VIEW dbo.v_OrderBasic AS
SELECT
  o.OrderID,
  c.Name        AS CustomerName,
  s.CompanyName AS ShipperName,
  o.OrderDate
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
LEFT JOIN dbo.Shippers s ON s.ShipperID = o.ShippersID;
GO
```

Rysunek 64: Definicja widoku v_OrderBasic

Widok **v_OrderBasic** prezentuje podstawowe informacje o zamówieniach: identyfikator zamówienia, nazwę klienta, nazwę przewoźnika oraz datę złożenia zamówienia.

7.5. Widok v_OrderValue

```sql
CREATE OR ALTER VIEW dbo.v_OrderValue AS
SELECT
  od.OrderID,
  CAST(SUM(od.UnitPrice * od.Quantity) AS DECIMAL(18,2)) AS ItemsTotal,
  SUM(od.Quantity)                                       AS ItemsQuantity
FROM dbo.OrderDetails od
GROUP BY od.OrderID;
GO
```

Rysunek 65: Definicja widoku v_OrderValue

Widok **v_OrderValue** przedstawia wartość zamówień wyliczaną na podstawie pozycji zamówienia. Umożliwia szybkie uzyskanie łącznej kwoty dla każdego zamówienia na bazie danych z tabeli **OrderDetails** (suma `UnitPrice * Quantity`).

7.6. Widok v_CustomerOrderHistory

```sql
CREATE OR ALTER VIEW dbo.v_CustomerOrderHistory AS
    SELECT
        c.CustomerID,
        c.Name as CustomerName,
        o.OrderID,
        o.OrderDate,
        o.DeliveryDate,
        o.Status as OrderStatus,
        o.Address as DeliveryAddress,
        o.City as DeliveryCity,
        o.Country,
        p.ProductName,
        od.UnitPrice,
        od.Quantity,
        (od.UnitPrice * od.Quantity) AS ItemTotal

    FROM Customers as c
    INNER JOIN Orders as o
        ON c.CustomerID = o.CustomerID
    INNER JOIN OrderDetails as od
                ON o.OrderID = od.OrderID
    INNER JOIN Products as p
                ON od.ProductID = p.ProductID;

GO
```

Rysunek 66: Definicja widoku v_CustomerOrderHistory

Widok **v_CustomerOrderHistory** agreguje szczegółowe dane dotyczące historii zakupów dokonywanych przez klientów. Tworzy spójny raport zawierający dane klienta, parametry logistyczne zamówienia oraz specyfikację zakupionych produktów wraz z ich wartością. Jest to kluczowy element dla działu obsługi klienta oraz analizy sprzedaży.

7.7. Widok v_OrderSchedule

```sql
CREATE OR ALTER VIEW dbo.v_OrderSchedule AS
SELECT
    o.OrderID,
    o.OrderDate,
    s.CompanyName AS ShipperName,
    oe.PredictedOrderDelivery,
    o.DeliveryDate AS ActualDeliveryDate,


    CASE
        WHEN o.DeliveryDate IS NOT NULL
        THEN DATEDIFF(DAY, oe.PredictedOrderDelivery, o.DeliveryDate)
    END AS DelayDays,


    CASE
        WHEN o.DeliveryDate IS NULL AND oe.PredictedOrderDelivery < GETDATE()
            THEN 'Opóźnione (w trakcie)'
        WHEN o.DeliveryDate IS NULL AND oe.PredictedOrderDelivery >= GETDATE()
            THEN 'W trakcie (na czas)'
        WHEN o.DeliveryDate IS NOT NULL AND o.DeliveryDate <= oe.PredictedOrderDelivery
            THEN 'Dostarczone na czas'
        WHEN oe.PredictedOrderDelivery IS NULL THEN 'Brak planu'
        ELSE 'Dostarczone z opóźnieniem'
    END AS DeliveryStatus,

    o.City,
    o.Country,
    c.Name AS CustomerName

FROM dbo.Orders o
INNER JOIN dbo.Customers c ON o.CustomerID = c.CustomerID
LEFT JOIN dbo.OrderExecution oe ON o.OrderID = oe.OrderID
LEFT JOIN dbo.Shippers s ON o.ShippersID = s.ShipperID;
GO
```

Rysunek 67: Definicja widoku v_OrderSchedule

Widok **v_OrderSchedule** pełni funkcję zaawansowanego raportu logistycznego, służącego do monitorowania terminowości dostaw. Porównuje on planowany termin realizacji (z tabeli OrderExecution) z rzeczywistą datą dostarczenia towaru, automatycznie wyliczając opóźnienia oraz nadając statusy tekstowe dla każdego zamówienia.

7.8. Widok v_ProductSalesStatistic

```sql
CREATE or ALTER VIEW dbo.v_ProductSalesStatistic AS
    SELECT
        p.ProductID,
        p.ProductName,
        p.Price as CurrentPrice,
        SUM(od.Quantity) as TotalSoldQuantity,
        SUM(od.Quantity * od.UnitPrice) as TotalRevenue,
        COUNT(DISTINCT o.OrderID) as NumberOfOrders,
        COUNT(Distinct o.CustomerID) as UniqueCustomers,
        MIN(o.OrderDate) as FirstSaleDate,
        MAX(o.OrderDate) as LastSaleDate,
        AVG(od.Quantity) as AverageQuantityPerOrder,
        CASE
            WHEN COUNT(o.OrderID) > 0
            THEN SUM(od.Quantity * od.UnitPrice) / COUNT(DISTINCT o.OrderID)
            ELSE 0
        END as AverageOrderValue

    FROM
        Products as p
    LEFT JOIN
            OrderDetails as od
            ON p.ProductID = od.ProductID
    LEFT JOIN
            Orders as o
            ON od.OrderID = o.OrderID
    GROUP BY p.ProductID, p.ProductName, p.Price
GO
```

Rysunek 68: Definicja widoku v_ProductSalesStatistic

Widok **v_ProductSalesStatistic** stanowi kompleksowe narzędzie analityczne, agregujące statystyki sprzedaży dla każdego produktu w ofercie.

8. Uprawnienia, użytkownicy systemu i funkcje użytkowników

System przewiduje podział użytkowników na role odpowiadające realnym stanowiskom w firmie. Każda rola posiada ograniczony zakres dostępu do danych oraz operacji, zgodny z wykonywanymi obowiązkami. Dzięki temu

ograniczana jest możliwość przypadkowej modyfikacji krytycznych danych oraz zachowana jest spójność procesu obsługi zamówień, produkcji i magazynu.

8.1. Rola admin (administrator)

```
create role admin;
grant all privileges to admin;
```

Rysunek 69: Uprawnienia roli admin

8.1.1. Użytkownik systemu

Użytkownik pełniący rolę **admin** ma pełny wgląd do całej bazy danych i możliwość wprowadzania dowolnych modyfikacji.

8.2. Rola employee (pracownik)

```
create role employee;

grant select, insert, update on Orders to employee;
grant select, insert, update on OrderDetails to employee;
grant select on Products to employee;

-- Row-Level Security (tylko swój rekord)
grant select, update on Employees to employee;

create security policy dbo.EmployeePolicy
add filter predicate dbo.fn_EmployeeAccess(EmployeeID) on dbo.Employees,
add block predicate dbo.fn_EmployeeAccess(EmployeeID) on dbo.Employees
after update with (state = on);
```

Rysunek 70: Uprawnienia roli employee

8.2.1. Użytkownik systemu

Użytkownik pełniący rolę **employee** ma możliwość wprowadzać oraz wyszukiwać rekordy z tabel **Orders** oraz **OrderDetails** a także wyszukiwać rekordy z tabeli **Products**. Ponadto, pracownik posiadający tę rolę może wyszukać oraz edytować rekord z tabeli **Employees** dotyczący siebie samego.

8.2.2. Funkcje użytkownika

- wprowadzanie, aktualizowanie oraz wyszukiwanie informacji dotyczących zamówień,
- wyszukiwanie informacji o produktach
- wyświetlanie i edytowanie danych dotyczących samego siebie

8.3. Zakres uprawnień

- **SELECT, INSERT, UPDATE**: **Orders, OrderDetails**
- **SELECT: Products**
- **SELECT, UPDATE**: **Employees** (tylko swój wiersz, wykorzystano Row-Level Security)

8.4. Rola warehouse (magazyn)

```sql
CREATE OR ALTER VIEW dbo.v_ActiveEmployeesOrders AS
SELECT
  e.EmployeeID,
  e.Name           AS EmployeeName,
  COUNT(o.OrderID) AS ActiveOrdersCount
FROM dbo.Employees e
LEFT JOIN dbo.Orders o
  ON o.EmployeeID = e.EmployeeID
 AND o.Status = 3
WHERE e.IsActive = 1
GROUP BY e.EmployeeID, e.Name;
GO
```

Rysunek 71: Uprawnienia roli warehouse

8.4.1. Użytkownik systemu

Użytkownik pełniący rolę **warehouse** odpowiada za obsługę stanów magazynowych części oraz produktów, a także za przygotowanie zamówień do realizacji. Pracownik magazynu

powinien mieć możliwość sprawdzenia, czy wymagane elementy są dostępne, oraz aktualizacji ilości po przyjęciach i wydaniach.

8.4.2. Funkcje użytkownika

- kontrola bieżących stanów magazynowych części i produktów,
- aktualizacja ilości w magazynie po realizacji zamówień lub dostawach,
- podgląd zamówień oraz ich szczegółów w celu kompletacji.

8.4.3. Zakres uprawnień

- **SELECT, UPDATE**: `InventoryProducts`, `InventoryParts`
- **SELECT**: `Products`, `Parts`, `Orders`, `OrderDetails`

Uprawnienia te pozwalają aktualizować wyłącznie dane magazynowe, przy jednoczesnym dostępie w trybie podglądu do danych potrzebnych do kompletacji zamówień.

8.5. Rola finance (finanse)

```sql
CREATE OR ALTER VIEW dbo.v_OrderBasic AS
SELECT
  o.OrderID,
  c.Name        AS CustomerName,
  s.CompanyName AS ShipperName,
  o.OrderDate
FROM dbo.Orders o
JOIN dbo.Customers c ON c.CustomerID = o.CustomerID
LEFT JOIN dbo.Shippers s ON s.ShipperID = o.ShippersID;
GO
```

Rysunek 72: Uprawnienia roli finance

8.5.1. Użytkownik systemu

Użytkownik pełniący rolę **finance** odpowiada za obsługę płatności i kontrolę rozliczeń. Powinien mieć dostęp do

informacji o zamówieniach oraz ich wartościach, aby weryfikować poprawność wpłat i status płatności.

8.5.2. Funkcje użytkownika

- rejestrowanie nowych płatności,
- aktualizacja statusu płatności,
- kontrola wartości zamówienia i weryfikacja, czy zostało opłacone,
- podgląd informacji o kliencie i produktach w kontekście rozliczeń.

8.5.3. Zakres uprawnień

- **SELECT, INSERT, UPDATE**: `Payments`
- **SELECT**: `Orders`, `OrderDetails`, `Customers`, `Products`

Uprawnienia finansowe umożliwiają zarządzanie płatnościami bez możliwości ingerencji w dane magazynowe ani w strukturę zamówień.

8.6. Rola reporting (raportowanie / podgląd)

```sql
CREATE OR ALTER VIEW dbo.v_OrderValue AS
SELECT
  od.OrderID,
  CAST(SUM(od.UnitPrice * od.Quantity) AS DECIMAL(18,2)) AS ItemsTotal,
  SUM(od.Quantity)                                       AS ItemsQuantity
FROM dbo.OrderDetails od
GROUP BY od.OrderID;
GO
```

Rysunek 73: Uprawnienia roli reporting

8.6.1. Użytkownik systemu

Rola **reporting** przeznaczona jest dla użytkowników wykonujących analizy i przygotowujących raporty (np. kadra zarządcza). Dostęp ogranicza się do odczytu danych oraz widoków ułatwiających raportowanie.

8.6.2. Funkcje użytkownika

- przegląd podstawowych danych o zamówieniach oraz ich wartości,
- analiza liczby aktywnych zamówień przypisanych do pracowników,
- tworzenie zestawień i raportów bez możliwości modyfikacji danych operacyjnych.

8.6.3. Zakres uprawnień

- **SELECT** na widokach:
  - `v_ActiveEmployeesOrders`
  - `v_OrderBasic`
  - `v_OrderValue`
- (opcjonalnie) **SELECT** na tabelach źródłowych wymaganych do raportów, jeśli widoki nie są wystarczające.

Takie rozdzielenie uprawnień zapewnia możliwość raportowania bez ryzyka modyfikacji danych operacyjnych.

8.7. Rola endUser (użytkownik)

```
CREATE ROLE endUser;

GRANT SELECT ON Products to endUser;

GRANT EXECUTE ON AddCustomer to endUser;
GRANT EXECUTE ON ChangeCustomerContact  to endUser;
GRANT EXECUTE ON ChangeCustomerData to endUser;
```

Rysunek 74: Uprawnienia roli endUser

Użytkownik pełniący rolę **endUser** reprezentuje klienta zewnętrznego (np. osobę odwiedzającą stronę internetową sklepu). Rola ta posiada uprawnienia niezbędne do zapoznania się z ofertą handlową oraz samodzielnego zarządzania swoim

kontem. Ze względów bezpieczeństwa, modyfikacja danych klienta odbywa się wyłącznie poprzez dedykowane procedury, a nie bezpośredni dostęp do tabel.

8.8. Funkcje użytkownika

- przeglądanie katalogu dostępnych produktów,
- rejestracja w systemie (dodanie nowego klienta),
- aktualizacja własnych danych osobowych (np. zmiana nazwiska, adresu),
- aktualizacja danych kontaktowych (zmiana numeru telefonu)

8.8.1. Zakres uprawnień

- **SELECT**: `Products`
- **EXECUTE**: `AddCustomer`, `ChangeCustomerContact`, `ChangeCustomerData`

Użytkownik ma wgląd w asortyment, a modyfikacje wprowadza w sposób kontrolowany przez procedury, co zapobiega błędnym wpisom lub nieuprawnionemu dostępowi do danych innych klientów.

8.9. Rola logistics (logistyka)

```
CREATE ROLE logistics;

GRANT SELECT, UPDATE ON dbo.Orders TO logistics;
GRANT SELECT ON dbo.Customers TO logistics;
GRANT SELECT, INSERT, UPDATE ON dbo.Shippers TO logistics;
GRANT SELECT, UPDATE ON dbo.OrderExecution TO logistics;
```

Rysunek 75: Uprawnienia roli logistics

Użytkownik pełniący rolę **logistics** odpowiada za proces wysyłki zamówień oraz zarządzanie dostawcami. Jego zadaniem

jest przypisanie odpowiedniego kuriera do zamówienia, planowanie terminów dostaw oraz monitorowanie statusu przesyłek.

8.9.1. Funkcje użytkownika

- zarządzanie bazą firm kurierskich (dodawanie nowych przewoźników, aktualizacja danych kontaktowych),
- planowanie i nadzorowanie procesu realizacji zamówienia (harmonogram wysyłek),
- podgląd danych adresowych klientów w celu organizacji transportu

8.9.2. Zakres uprawnień

- **SELECT**, **INSERT**, **UPDATE**: `Shippers`, `OrderExecution`
- **SELECT**, **UPDATE**: `Orders`
- **SELECT**: `Customers`

Dzięki takiemu skonfigurowaniu uprawnień, logistyk ma pełną autonomię w organizacji transportu, jednocześnie nie mogąc ingerować w treść handlową zamówienia (np. nie może usunąć zamówienia ani dodać do niego produktów.